KJV TECH WEB DEVELOPMENT MASTERCLASS

A Practical Guide to HTML5, CSS3, JavaScript, Git & GitHub

Author: Kayode J.V

Version: 1.0

KJV TECH

Web Development Masterclass

A Practical Guide to HTML5, CSS3, JavaScript, Git & GitHub

Author: Kayode J.V
Version: 1.0

PREFACE

Welcome to the **KJV Tech Web Development Masterclass**.

This manual is designed to take complete beginners from having little or no programming knowledge to being able to build modern, responsive, and dynamic websites.

The course emphasizes practical learning. Every topic includes explanations, examples, hands-on exercises, assignments, and projects. By the end of the programme, students will have built a portfolio showcasing their skills.

Whether your goal is to become a freelance web developer, secure an internship, launch a tech career, or create websites for your own business, this manual provides a structured path to achieving it.

Table of Contents

MODULE 1: INTRODUCTION TO WEB DEVELOPMENT

MODULE 2: HTML5

MODULE 3: CSS3

MODULE 4: JAVASCRIPT

MODULE 5: GIT & GITHUB

MODULE ONE
INTRODUCTION TO WEB DEVELOPMENT
CHAPTER ONE
UNDERSTANDING THE INTERNET

Learning Objectives

At the end of this chapter, students should be able to:

- Define the Internet.
- Explain how websites work.
- Differentiate between a website and a web application.
- Identify the major components involved when accessing a website.

What is the Internet?

The Internet is a global network of interconnected computers and devices that communicate with one another using standardized protocols. It enables people around the world to share information, communicate, learn, conduct business, and access digital services.

When you open a website such as Google or YouTube, your device sends a request through the Internet to a server. The server processes the request and sends the requested web page back to your browser for display.

What is a Website?

A website is a collection of related web pages stored on a web server and accessed through a web browser using a domain name or web address.

Examples include:

- Educational websites
- Church websites
- Business websites
- E-commerce websites
- News websites

- Personal portfolios

What is a Web Browser?

A web browser is software used to access and display websites.

Examples include:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Safari
- Opera

Class Activity

Open your preferred browser and visit:

1. google.com
2. wikipedia.org
3. github.com

Observe the differences in their design and discuss the purpose of each website.

Assignment

Answer the following questions:

1. Define the Internet.
2. What is a website?
3. Name five examples of web browsers.
4. Explain the difference between a website and a web application.
5. Why is the Internet important in modern society?

CHAPTER 2

HOW WEBSITES WORK

Learning Objectives

At the end of this chapter, students should be able to:

- Explain how a website works.
- Understand the relationship between a client and a server.
- Define a web server.
- Explain the purpose of a domain name and web hosting.
- Understand the basic flow of an HTTP request.
- Differentiate between HTTP and HTTPS.

Introduction

Every time you open a website like Google, Facebook, or Amazon, many things happen behind the scenes within seconds. Although it seems like the website appears instantly, several technologies work together to deliver the page to your screen.

Understanding how websites work is one of the most important foundations of web development.

2.1 What Happens When You Visit a Website?

Suppose you type: www.google.com into your web browser. The following steps happen:

Step 1: You Enter the Website Address

You type the website address (called the **URL(Uniform Resource Locator)**) into your browser.

Example: https://www.google.com

Step 2: DNS Finds the Website

Computers do not understand names like **google.com**.

Instead, they communicate using **IP addresses**.

Example: 142.250.183.14

A **DNS (Domain Name System)** translates the website name into its IP address.

Think of DNS as the Internet's phonebook.

Step 3: Your Browser Sends a Request

After finding the IP address, your browser sends a request to Google's web server asking for the webpage.

This request uses the **HTTP** or **HTTPS** protocol.

Step 4: The Server Processes the Request

The server receives your request.

It may:

- Retrieve files
- Access a database
- Run PHP or another backend language
- Process user information

Step 5: The Server Responds

The server sends back:

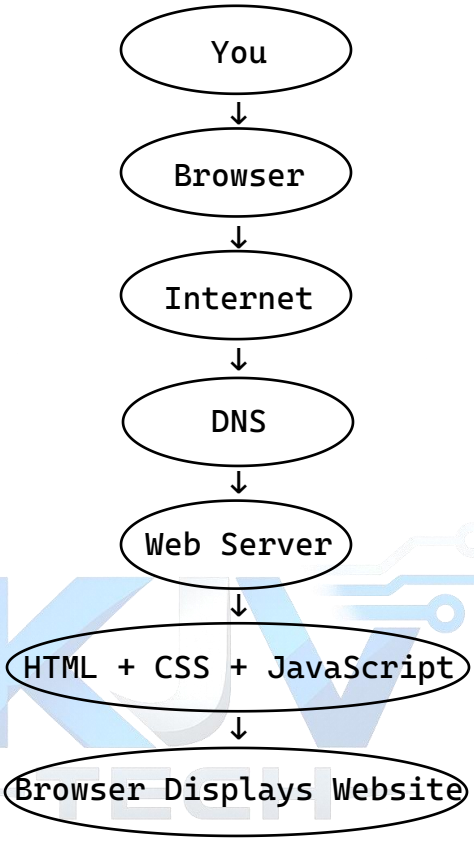
- HTML
- CSS
- JavaScript
- Images
- Videos

to your browser.

Step 6: The Browser Displays the Website

Your browser combines everything together and displays the webpage.

Illustration



2.2 Client and Server

One of the most important concepts in web development is understanding the difference between a **client** and a **server**.

Client

A **client** is any device or software that requests information.

Examples include:

- Laptop
- Desktop
- Smartphone
- Tablet

Browsers such as:

- Google Chrome
- Microsoft Edge
- Firefox
- Safari

are examples of client software.

Server

A **server** is a powerful computer that stores websites and responds to requests from users.

Popular web servers include:

- Apache
- Nginx
- LiteSpeed
- Microsoft IIS

Example

Imagine ordering food in a restaurant. You are the **client**. The kitchen is the **server**. You place an order (request). The kitchen prepares the food. The waiter delivers the food (response).

2.3 Static vs Dynamic Websites

Static Website

A static website always shows the same content.

It is built mainly using:

- HTML
- CSS
- JavaScript (optional)

Examples:

- Personal portfolio
- Company profile

- Church landing page

Advantages:

- Fast
- Easy to build
- Secure

Dynamic Website

A dynamic website changes based on the user or stored data.

Built using:

- PHP
- Node.js
- Python
- Java
- ASP.NET

Examples:

- Facebook
- Instagram
- Amazon
- Online banking
- School portals

Advantages:

- Interactive
- Stores user data
- Can process forms and logins

2.4 Domain Name

A **domain name** is the human-readable name of a website.

Examples: google.com, amazon.comopenai.com, kjvtech.com

Without domain names, users would have to remember IP addresses.

2.5 Web Hosting

Web hosting is a service that stores website files and makes them accessible on the Internet.

Hosting companies provide:

- Storage space
- Security
- Databases
- Bandwidth
- Backups

Examples of hosting providers include Hostinger, Namecheap, Bluehost, InfinityFree (free hosting), and Render (for many web applications).

2.6 URL (Uniform Resource Locator)

A URL is the complete address used to locate a resource on the Internet.

Example: https://www.example.com/about.html

Parts of the URL:

Part	Description
https	Protocol
www	Subdomain
example.com	Domain name
about.html	Specific webpage

2.7 HTTP vs HTTPS

HTTP

HyperText Transfer Protocol

Used for transferring webpages.

http://example.com

Data sent using HTTP is not encrypted.

HTTPS

HyperText Transfer Protocol Secure

https://example.com

HTTPS encrypts data between the browser and the server, making it much more secure. Modern websites should always use HTTPS.

2.8 Frontend and Backend

Frontend

The frontend is everything users see and interact with in a browser.

Languages:

- HTML
- CSS
- JavaScript

Backend

The backend runs on the server.

Languages include:

- PHP
- Java
- Python
- Node.js

Responsibilities:

- User authentication

- Database operations
- Payment processing
- Business logic

Key Terms

Term	Meaning
Browser	Software used to access websites
Client	Device or application requesting data
Server	Computer that stores and serves website files
Domain Name	Human-readable website address
Hosting	Service that stores website files
DNS	Converts domain names into IP addresses
HTTP	Protocol for transferring webpages
HTTPS	Secure version of HTTP
URL	Address of a webpage

Hands-on Activity

1. Open your browser and visit five different websites.
2. Observe whether the address begins with **http://** or **https://**.
3. Identify the domain name for each website.
4. Discuss whether each site appears to be static or dynamic, and explain your reasoning.

Review Questions

1. Explain the process that occurs when you type a website address into your browser.
2. What is the difference between a client and a server?
3. What is the purpose of DNS?

4. Differentiate between a static website and a dynamic website.
5. What is a domain name?
6. What is web hosting?
7. State three differences between HTTP and HTTPS.
8. Identify the parts of the following URL:

https://www.example.com/contact.html

9. List three examples of backend programming languages.
10. Why is HTTPS recommended for modern websites?

Practical Assignment

Project: Analyze a Website

Choose any **three websites** and answer the following for each:

- Website name
- URL
- Is it static or dynamic?
- Does it use HTTP or HTTPS?
- What do you think its main purpose is?
- Identify at least three visible frontend elements (e.g., navigation bar, images, forms, buttons).

Prepare your findings in a one-page report and be ready to present them in the next class.

CHAPTER 3

FRONTEND VS BACKEND DEVELOPMENT

Learning Objectives

At the end of this chapter, students should be able to:

- Define frontend development.
- Define backend development.
- Explain the responsibilities of frontend and backend developers.
- Differentiate between frontend, backend, and full-stack development.
- Identify the technologies used in each area.
- Understand how the frontend and backend communicate.

Introduction

Every website or web application is divided into two major parts:

1. **The Frontend**
2. **The Backend**

Think of a website as a restaurant.

- The dining area, menu, tables, and waiters represent the **frontend** because customers can see and interact with them.
- The kitchen, chefs, and food preparation represent the **backend** because customers cannot see them, but they make everything work.

Both parts must work together to provide a complete experience for users.

3.1 What is Frontend Development?

Frontend development refers to the part of a website or web application that users can **see and interact with**.

Everything displayed in a browser is part of the frontend.

Examples include:

- Navigation menus
- Images
- Buttons
- Forms
- Text
- Videos
- Animations
- Layouts

A frontend developer focuses on creating visually appealing, responsive, and user-friendly interfaces.

1. **Frontend Technologies**

➢ **HTML (HyperText Markup Language)**

HTML provides the structure of a webpage.

Example:

```
<h1>Welcome to KJV Tech</h1>
```

➢ **CSS (Cascading Style Sheets)**

CSS is used to style HTML elements.

Example:

```
h1{
color: blue;
}
```

➢ **JavaScript**

JavaScript adds interactivity.

Examples:

- Pop-up messages
- Form validation

- Image sliders
- Interactive menus
- Calculators

Example:

```
alert("Welcome to KJV Tech!");
```

❖ **RESPONSIBILITIES OF A FRONTEND DEVELOPER**

A frontend developer is responsible for:

- Designing webpage layouts
- Making websites responsive
- Improving user experience
- Writing HTML
- Styling with CSS
- Programming interactions with JavaScript
- Testing websites across different devices and browsers

3.2 WHAT IS BACKEND DEVELOPMENT?

Backend development is everything that happens **behind the scenes**.

Users do not directly see the backend, but they interact with it whenever they log in, register, search for information, or submit forms.

The backend processes requests, performs calculations, stores information, and sends responses to the frontend.

Responsibilities of a Backend Developer

A backend developer is responsible for:

- Writing server-side code

- Managing databases
- Processing user requests
- User authentication
- Payment processing
- Sending emails
- API development
- Data security

Backend Programming Languages

Popular backend languages include:

- PHP
- Java
- Python
- JavaScript (Node.js)
- C#
- Ruby
- Go

In this course, we will use **PHP** because it is beginner-friendly, widely supported, and integrates well with MySQL.

3.3 WHAT IS A DATABASE?

A database is a structured collection of data stored electronically.

Websites use databases to store information such as:

- User accounts
- Passwords (stored securely using hashing)
- Products
- Orders
- Student records
- Blog posts

- Messages

Examples of database management systems include:

- MySQL
- PostgreSQL
- SQLite
- Microsoft SQL Server
- Oracle Database

3.4 HOW FRONTEND AND BACKEND WORK TOGETHER

Imagine you are logging into a student portal.

Step 1

You enter:

- Username
- Password

This is done on the **frontend**.

↓

Step 2

You click the **Login** button.

↓

Step 3

The frontend sends your information to the backend.

↓

Step 4

The backend checks the database.

↓

Step 5

If your credentials are correct, the backend sends a success response.

↓

Step 6

The frontend displays your dashboard.

Workflow Illustration

User

↓

Frontend

(HTML + CSS + JavaScript)

↓

Backend

(PHP)

↓

Database

(MySQL)

↓

Backend

↓

Frontend

↓

User sees the result

3.5 What is Full-Stack Development?

A full-stack developer can build both the frontend and the backend of a web application.

They understand:

- HTML

- CSS
- JavaScript
- PHP
- Databases
- Git
- Deployment

This course is designed to introduce students to full-stack web development by covering both client-side and server-side technologies.

3.6 Real-World Example

Consider an online shopping website.

Frontend

Users can:

- Browse products
- Search for items
- Add products to a cart
- Fill out checkout forms

Backend

The system:

- Checks product availability
- Calculates totals
- Processes payments
- Stores orders
- Sends confirmation emails

Both sides work together to complete a purchase.

3.7 Skills Required for a Web Developer

Technical Skills

- HTML5
- CSS3

- JavaScript
- PHP
- MySQL
- Git & GitHub
- Debugging
- Responsive Design

Soft Skills

- Problem-solving
- Communication
- Time management
- Teamwork
- Continuous learning
- Attention to detail

Comparison Table

Feature	Frontend	Backend
Visible to users	Yes	No
Runs on	Browser	Server
Main Purpose	User Interface	Business Logic & Data Processing
Languages	HTML, CSS, JavaScript	PHP, Java, Python, Node.js, C#
Interacts with Database	No (directly)	Yes

Career Paths

After completing this course, students may pursue roles such as:

- Frontend Developer
- Backend Developer

- Full-Stack Developer
- WordPress Developer
- PHP Developer
- Freelance Web Developer
- Technical Support Engineer
- Junior Software Developer

Best Practices

As a beginner:

- Write clean, readable code.
- Use meaningful file and folder names.
- Test your work regularly.
- Comment complex sections of your code.
- Keep learning through practice.
- Use Git to track changes to your projects.

Key Terms

Term	Meaning
Frontend	The visible part of a website that users interact with
Backend	The server-side part of a website that processes requests
Full-Stack Developer	A developer who works on both frontend and backend
Database	A structured collection of data
User Interface (UI)	The visual elements users interact with
Server-Side	Code executed on the server
Client-Side	Code executed in the user's browser

Hands-on Activity

1. Visit **www.amazon.com** or another e-commerce website.
2. Identify at least **five frontend elements** you can see (e.g., navigation bar, search box, product cards).
3. List **five backend operations** you think happen when you:
    - Log in
    - Search for a product
    - Add an item to your cart
    - Complete a purchase

Discuss your answers with the class.

Review Questions

1. Define frontend development.
2. Define backend development.
3. List three frontend technologies.
4. List four backend programming languages.
5. What is the role of a database in web development?
6. Explain the difference between frontend and backend.
7. Who is a full-stack developer?
8. What happens after a user clicks the "Login" button on a website?
9. Give three examples of websites that rely heavily on backend development.
10. Why are soft skills important for web developers?

Practical Assignment

Mini Project: Plan a Website

Choose one of the following:

- School Website
- Church Website

- Restaurant Website
- Online Store
- Personal Portfolio

Create a simple document that includes:

1. The purpose of the website.
2. Five frontend features (what users will see and interact with).
3. Five backend functions (what the server will do behind the scenes).
4. A list of the technologies you would use (HTML, CSS, JavaScript, PHP, MySQL, etc.).

Submission: 2–3 pages, typed or neatly written, to be discussed in the next class.

KJV TECH

CHAPTER 4:
ESSENTIAL DEVELOPMENT TOOLS

Learning Objectives

At the end of this chapter, students should be able to:

- Identify the essential tools required for web development.
- Install and configure Visual Studio Code (VS Code).
- Understand the purpose of a web browser in development.
- Explain the role of XAMPP in PHP development.
- Understand the basics of Git and GitHub.
- Organize project files using a professional folder structure.

Introduction

Before writing your first line of code, you need the right tools. Professional web developers use specialized software to write, test, debug, and deploy websites.

Think of a web developer as a carpenter. Just as a carpenter needs a hammer, saw, and measuring tape, a web developer needs tools like a code editor, browser, and web server.

By the end of this chapter, your computer will be fully prepared for web development.

4.1 Code Editors

A **code editor** is software used to write and edit programming code.

Unlike simple text editors such as Notepad, code editors provide features that make coding easier and more efficient.

Features of a Good Code Editor

- Syntax highlighting
- Auto-completion
- Error detection
- Code formatting

- Extensions
- Integrated terminal
- Search and replace

4.2 Visual Studio Code (VS Code)

Visual Studio Code (VS Code) is one of the most popular code editors among web developers. It is free, lightweight, and supports many programming languages through extensions.

Why We Use VS Code

- Free to download
- Beginner-friendly
- Fast and lightweight
- Supports HTML, CSS, JavaScript, PHP, and many other languages
- Built-in terminal
- Large extension marketplace

Installing VS Code

1. Download VS Code from the official website.
2. Run the installer.
3. Accept the license agreement.
4. Choose the installation location.
5. Enable:
    - Add to PATH
    - Create Desktop Icon
    - Open with Code
6. Finish the installation.

4.3 Useful VS Code Extensions

Extensions add extra features to VS Code.

Recommended extensions:

Extension	Purpose
Live Server	Opens websites in a browser and refreshes automatically
Prettier	Formats code automatically
Auto Rename Tag	Renames opening and closing HTML tags together
Auto Close Tag	Automatically closes HTML tags
HTML CSS Support	Improves HTML and CSS coding
IntelliCode	AI-assisted code suggestions
PHP Intelephense	PHP code completion and error checking

4.4 Web Browsers

A browser displays your website and helps you test it.

Recommended browsers:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Google Chrome Developer Tools

Developer Tools (DevTools) help you inspect and debug web pages.

You can:

- Inspect HTML elements
- Modify CSS temporarily
- View JavaScript errors
- Monitor network requests
- Test responsive layouts

Opening DevTools

Windows:

F12

or

Ctrl + Shift + I

4.5 File Explorer

Keeping files organized is an important habit for every developer.

Example project structure:

```
MyFirstWebsite/
                │
                ├── index.html
                ├── about.html
                ├── contact.html
                │
                ├── css/
                │     └── style.css
                │
                ├── js/
                │     └── script.js
                │
                ├── images/
                │
                └── assets/
```

This structure makes projects easier to manage as they grow.

4.6 XAMPP

When developing websites with PHP and MySQL, you need a local server.

XAMPP is a free software package that provides:

- Apache Web Server
- MySQL (MariaDB)

- PHP
- phpMyAdmin

This allows you to build and test websites on your own computer before publishing them online.

Installing XAMPP

1. Download XAMPP.
2. Install the software.
3. Open the XAMPP Control Panel.
4. Start:
    - Apache
    - MySQL

If both services show a green status, your local server is running.

Testing XAMPP

Open your browser and enter:

http://localhost

If the XAMPP dashboard appears, the installation was successful.

4.7 Git

Git is a **Version Control System (VCS)**.

It keeps track of changes made to your code over time.

Benefits of Git

- Tracks code history
- Allows collaboration
- Enables reverting to previous versions
- Supports branching and merging
- Widely used in the software industry

4.8 GitHub

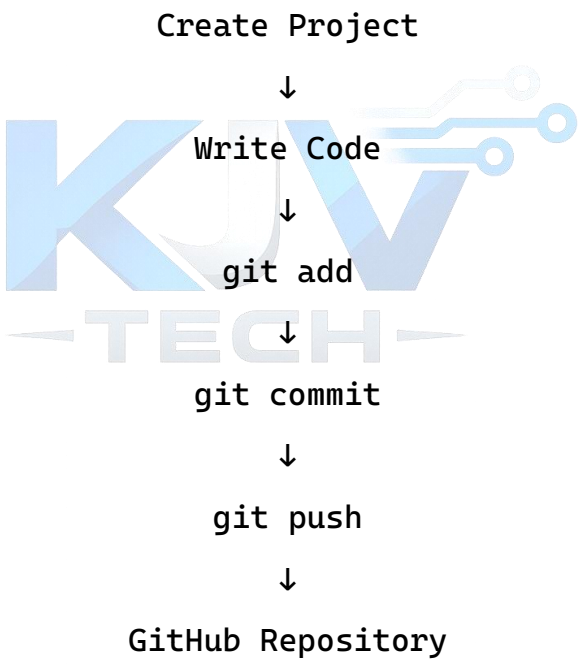
GitHub is an online platform for hosting Git repositories.

Developers use GitHub to:

- Store code online
- Collaborate with teams
- Share open-source projects
- Build a professional portfolio
- Showcase projects to employers

4.9 Basic Git Workflow

A typical Git workflow is:



We'll learn these commands in detail in a later module.

4.10 Recommended Software for This Course

Software	Purpose
Visual Studio Code	Code Editor
Google Chrome	Browser
XAMPP	PHP & MySQL Server

Software	Purpose
Git	Version Control
GitHub	Online Code Repository
Notepad++ (Optional)	Lightweight Text Editor

4.11 Good Development Practices

Develop these habits from day one:

- Create a separate folder for every project.
- Use meaningful file names (e.g., index.html, style.css).
- Save your work frequently.
- Back up important projects.
- Write clean and readable code.
- Test your website after every major change.
- Comment your code when necessary.
- Keep your project folders organized.

Key Terms

Term	Meaning
Code Editor	Software used to write code
VS Code	A popular source code editor
Extension	An add-on that provides extra functionality
Browser	Software for viewing websites
DevTools	Built-in browser tools for debugging
XAMPP	Local server software for PHP and MySQL
Git	Version control system
GitHub	Cloud platform for Git repositories

Term	Meaning
Repository	A storage location for project files and code

Hands-on Activity

1. Install Visual Studio Code.
2. Install Google Chrome (if not already installed).
3. Install XAMPP.
4. Install Git.
5. Create a GitHub account.
6. Create a folder named **KJV-Web-Development**.
7. Inside the folder, create the following structure:

```
KJV-Web-Development/
        │
        ├── Module1/
        ├── Module2/
        ├── Projects/
        ├── Assignments/
        └── Notes/
```

Review Questions

1. What is a code editor?
2. Why is Visual Studio Code recommended for beginners?
3. List five useful VS Code extensions.
4. What are Chrome Developer Tools used for?
5. What is XAMPP?
6. Name the four main components included in XAMPP.
7. What is Git?
8. What is GitHub?
9. Why is version control important in software development?

10. Why should developers organize project files properly?

Practical Assignment

Project: Set Up Your Development Environment

Complete the following tasks:

1. Install Visual Studio Code.
2. Install the **Live Server** extension.
3. Install XAMPP and start **Apache** and **MySQL**.
4. Create a GitHub account.
5. Create your first repository named **web-development-practice**.
6. Create a folder named **MyFirstWebsite** with the following structure:

```
MyFirstWebsite/
│
├── index.html
├── css/
│   └── style.css
├── js/
│   └── script.js
├── images/
└── README.md
```

Take screenshots of:

- Your VS Code window
- XAMPP Control Panel (Apache and MySQL running)
- Your GitHub repository
- Your project folder structure

Submit these screenshots along with a brief explanation of each tool and its purpose.

CHAPTER 5
INSTALLING AND USING VISUAL STUDIO CODE

Learning Objectives

At the end of this chapter, students should be able to:

- Explain what Visual Studio Code (VS Code) is.
- Download and install Visual Studio Code.
- Navigate the VS Code interface.
- Create and organize web development projects.
- Open and manage folders and files.
- Use the Integrated Terminal.
- Install useful extensions.
- Customize VS Code for better productivity.
- Apply keyboard shortcuts to improve coding efficiency.

INTRODUCTION

Every professional developer needs a reliable code editor to write, edit, and manage code efficiently. While many code editors exist, **Visual Studio Code (VS Code)** has become one of the most popular choices among web developers because it is free, lightweight, customizable, and supports almost every programming language.

In this chapter, you will learn how to use VS Code as your primary development environment. Mastering this tool will help you write code faster, stay organized, and prepare for professional software development.

5.1 WHAT IS VISUAL STUDIO CODE?

Visual Studio Code (VS Code) is a free source-code editor developed by Microsoft.

It supports many programming languages, including:

- HTML
- CSS

- JavaScript
- PHP
- Python
- Java
- C#
- C++
- TypeScript

VS Code is not just a text editor—it includes powerful tools such as debugging, version control integration, intelligent code completion, and an extension marketplace.

5.2 Features of Visual Studio Code

Some of the features that make VS Code popular include:

- Syntax Highlighting
- Auto-completion (IntelliSense)
- Built-in Terminal
- Integrated Git Support
- Extension Marketplace
- Debugging Tools
- Multiple Themes
- Workspace Management
- File Explorer
- Split Editor

These features help developers write clean, efficient, and organized code.

5.3 Downloading and Installing VS Code

Step 1

Visit the official Visual Studio Code website.

Step 2

Download the version compatible with your operating system (Windows, macOS, or Linux).

Step 3

Run the installer.

Step 4

During installation, enable the following options:

- ✓ Add to PATH
- ✓ Register Code as an editor
- ✓ Add "Open with Code" to the context menu
- ✓ Create a Desktop Icon

Step 5

Complete the installation and launch VS Code.

5.4 Exploring the VS Code Interface

When you open VS Code, you will see several sections.

1. Activity Bar

Located on the left side.

It contains icons for:

- Explorer
- Search
- Source Control
- Run & Debug
- Extensions

2. Explorer

The Explorer displays your project folders and files.

It allows you to:

- Create files
- Create folders

- Rename files
- Delete files
- Organize projects

3. Editor Area

The Editor Area is where you write and edit code.

You can open multiple files simultaneously using tabs.

4. Status Bar

Located at the bottom.

It displays:

- Programming language
- Line and column numbers
- Git branch
- Notifications

5. Integrated Terminal

The Integrated Terminal allows you to execute commands without leaving VS Code.

Open it using:

Menu

Terminal → New Terminal

or press:

Ctrl + `

(Commonly called **Control + Backtick**.)

5.5 Creating Your First Project

Let's create your first project.

Step 1

Create a folder called:

MyFirstWebsite

Step 2

Open VS Code.

Step 3

Click:

File → Open Folder

Select the **MyFirstWebsite** folder.

5.6 Creating Files

Inside your project folder, create the following files:

```
MyFirstWebsite/
        index.html
    css/
        style.css
    js/
        script.js
    images/
```

This folder structure is considered good practice because it separates HTML, CSS, JavaScript, and images into dedicated directories.

5.7 Understanding File Extensions

Each file type has a specific extension.

Extension	Purpose
.html	HTML files
.css	CSS stylesheets
.js	JavaScript files
.php	PHP files
.jpg	Image files
.png	Image files

Extension	Purpose
.jpeg	Image files

Using the correct extension ensures that the operating system and browser know how to handle the file.

5.8 Installing VS Code Extensions

Extensions add new features to VS Code.

To install an extension:

1. Click the **Extensions** icon.
2. Search for the extension.
3. Click **Install**.

Recommended Extensions

Live Server

Automatically refreshes the browser whenever you save changes.

Prettier

Formats your code neatly.

Auto Rename Tag

Automatically renames matching HTML tags.

Auto Close Tag

Automatically closes HTML tags.

IntelliCode

Provides AI-assisted code suggestions.

PHP Intelephense

Improves PHP development with code completion and error detection.

5.9 Using Live Server

Live Server allows you to preview your website in a browser with automatic updates whenever you save your files.

Steps

1. Open **index.html**.
2. Right-click anywhere inside the file.
3. Click:

Open with Live Server

Your browser will open automatically.

Every time you press **Ctrl + S**, the page refreshes automatically.

5.10 Integrated Terminal

Instead of opening Command Prompt separately, VS Code includes a built-in terminal.

Common uses:

- Running Git commands
- Running PHP
- Running Node.js
- Navigating folders
- Installing packages

Example:

```
git init
```

or

```
php -S localhost:8000
```

5.11 Customizing VS Code

You can personalize VS Code by changing:

- Theme
- Icons
- Font Size
- Font Family
- Auto Save

- Word Wrap

To access settings:

File → Preferences → Settings

or

Ctrl + ,

5.12 Essential Keyboard Shortcuts

Shortcut	Action
Ctrl + N	New File
Ctrl + S	Save File
Ctrl + Shift + S	Save As
Ctrl + P	Open File
Ctrl + F	Find
Ctrl + H	Replace
Ctrl + /	Comment Code
Ctrl + Z	Undo
Ctrl + Y	Redo
Ctrl + C	Copy
Ctrl + V	Paste
Ctrl + X	Cut
Ctrl + A	Select All
Ctrl + Shift + P	Command Palette
Ctrl + `	Open Terminal

Learning these shortcuts will significantly improve your coding speed and efficiency.

5.13 Best Practices When Using VS Code

- Keep one project per folder.
- Use descriptive file and folder names.
- Save your work frequently.
- Format your code regularly.
- Install only the extensions you need.
- Back up your projects using GitHub.
- Organize related files into dedicated folders.
- Close unused tabs to keep your workspace tidy.

Hands-on Activity

Complete the following tasks:

1. Install Visual Studio Code.
2. Install the **Live Server** extension.
3. Create a folder named **WebDevPractice**.
4. Inside the folder, create:
    - index.html
    - css/style.css
    - js/script.js
    - images/
5. Open the folder in VS Code.
6. Explore the Activity Bar, Explorer, Editor Area, and Terminal.
7. Open the Integrated Terminal and practice navigating between folders.

Chapter Summary

In this chapter, you learned:

- What Visual Studio Code is and why it is widely used.
- How to install and configure VS Code.
- The purpose of the main sections of the VS Code interface.
- How to create and organize project folders and files.

- How to install useful extensions such as Live Server and Prettier.
- How to use the Integrated Terminal.
- Essential keyboard shortcuts to improve productivity.

These skills will form the foundation of your coding workflow throughout this course.

Review Questions

1. What is Visual Studio Code?
2. Mention five features of VS Code.
3. What is the purpose of the Explorer panel?
4. Explain the function of the Integrated Terminal.
5. List four useful VS Code extensions for web development.
6. Why is Live Server useful?
7. Why should developers organize project files into folders?
8. What is the difference between a file and a folder?
9. State five keyboard shortcuts and their functions.
10. Explain why VS Code is one of the most popular code editors.

Practical Assignment

Project: Setting Up Your Professional Workspace

Create the following folder structure in VS Code:

```
Web Development/
                │
                ├── Module 1/
                ├── Assignments/
                ├── Projects/
                ├── Resources/
                ├── HTML/
                ├── CSS/
```

```
├── JavaScript/
└── PHP/
```

Then:

1. Install **Live Server**, **Prettier**, and **Auto Rename Tag**.
2. Create a new file named index.html inside the **HTML** folder.
3. Open the file using **Live Server**.
4. Take screenshots of:
    - The VS Code interface
    - Your folder structure
    - The Extensions panel
    - The Live Server running in your browser

Prepare a one-page report explaining the purpose of each tool you installed and how it contributes to efficient web development.

CHAPTER 6

INTRODUCTION TO HTML

Learning Objectives

At the end of this chapter, students should be able to:

- Define HTML and explain its importance in web development.
- Differentiate between a markup language and a programming language.
- Explain the structure of an HTML document.
- Create and save an HTML file.
- Write simple HTML code using common tags.
- Display an HTML page in a web browser.
- Apply HTML best practices when writing code.

Introduction

Every website on the Internet begins with **HTML**. Whether you are visiting Google, Facebook, Amazon, or a personal portfolio, HTML provides the structure that browsers use to display content.

Think of HTML as the **skeleton of the human body**. Just as a skeleton gives shape and support to the body, HTML gives structure to a webpage. Without HTML, there would be no headings, paragraphs, images, buttons, or links.

In this chapter, you will write your very first HTML code and learn how webpages are structured.

6.1 WHAT IS HTML?

HTML stands for **HyperText Markup Language**.

Let's break it down:

- **HyperText** refers to text that contains links to other web pages.

- **Markup** means using tags to describe and organize content.
- **Language** refers to the standard rules and syntax used to write HTML documents.

HTML tells the browser:

- What content to display.
- How the content is organized.
- The relationship between different elements on a webpage.

6.2 Why Do We Use HTML?

HTML is used to:

- Create web pages.
- Organize webpage content.
- Display text and images.
- Create hyperlinks.
- Design forms.
- Embed videos and audio.
- Structure information for browsers and search engines.

Without HTML, a browser cannot understand how to display webpage content.

6.3 Is HTML a Programming Language?

No.

HTML is **not** a programming language.

HTML cannot:

- Perform calculations.
- Make decisions.
- Repeat tasks using loops.
- Store variables.

Instead, HTML is a **markup language** used to describe the structure of content.

Programming languages such as JavaScript, PHP, Python, and Java provide functionality and logic, while HTML provides structure.

6.4 Creating Your First HTML File

Step 1

Open **Visual Studio Code**.

Step 2

Create a new folder named:

MyFirstWebsite

Step 3

Inside the folder, create a file named:

index.html

The .html extension tells the browser that this is an HTML document.

6.5 The Basic Structure of an HTML Document

Every HTML document follows a standard structure.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Website</title>
</head>
<body>
    <h1>Welcome to KJV Tech</h1>
    <p>This is my first HTML webpage.</p>
</body>
</html>
```

6.6 Understanding the HTML Structure

<!DOCTYPE html>

Declares that the document uses HTML5.

It helps the browser display the webpage correctly.

<html>

This is the root element.

Every HTML element is placed inside the <html> tag.

<head>

Contains information about the webpage.

Examples include:

- Title
- Character encoding
- CSS links
- JavaScript links
- Metadata

The content inside the <head> section is generally **not displayed** on the webpage.

<title>

Defines the text shown on the browser tab.

Example:

<title>KJV Tech</title>

<body>

Contains everything users can see on the webpage.

Examples include:

- Headings
- Paragraphs
- Images
- Buttons

- Links
- Tables
- Forms

6.7 HTML Tags

HTML uses **tags** to create webpage elements.

Example:

```
<h1>Hello World</h1>
```

Here:

- <h1> is the opening tag.
- </h1> is the closing tag.
- **Hello World** is the content.

6.8 HTML Elements

An HTML element consists of:

- Opening tag
- Content
- Closing tag

Example:

```
<p>I love web development.</p>
```

Everything together forms a paragraph element.

6.9 HTML Attributes

Attributes provide additional information about HTML elements.

Example:

```
<img src="student.jpg" alt="Student Picture">
```

In this example:

- src specifies the image location.

- alt provides alternative text if the image cannot be displayed.

6.10 Common HTML Tags

Tag	Purpose
<html>	Root element
<head>	Page information
<title>	Browser tab title
<body>	Visible webpage content
<h1>–<h6>	Headings
<p>	Paragraph
 	Line break
<hr>	Horizontal line
<!-- -->	Comment

6.11 Saving and Viewing an HTML Page

There are two ways to view your webpage.

Method 1

Double-click the HTML file.

The browser opens automatically.

Method 2 (Recommended)

Use **Live Server**.

Steps:

1. Open the HTML file in VS Code.
2. Right-click inside the file.
3. Select:

Open with Live Server

The webpage opens automatically and refreshes whenever you save your file.

6.12 Writing Your First Webpage

Create the following webpage.

```
<!DOCTYPE html>
<html>
<head>
    <title>My First Webpage</title>
</head>
<body>
    <h1>Welcome to KJV Tech</h1>
    <p>My name is ……..</p>
    <p>I am learning Web Development.</p>
    <hr>
    <p>This is my first webpage.</p>
</body>
</html>
```

Save the file and open it using Live Server.

6.13 Best Practices

As you write HTML code:

- Use lowercase tag names.
- Indent nested elements properly.
- Close tags correctly.
- Save your work frequently.
- Use meaningful file names.
- Write clean and readable code.
- Keep your code organized.

Following these practices will make your code easier to read and maintain.

Common Mistakes Beginners Make

- Forgetting to close tags.
- Saving files as .txt instead of .html.
- Placing content outside the <body> element.
- Misspelling HTML tags.
- Using uppercase file extensions (e.g., .HTML) inconsistently.

Real-World Example

Imagine building a house:

- HTML is the framework and walls.
- CSS is the paint, decoration, and furniture.
- JavaScript is the electricity and automation.

All three technologies work together to create a complete website.

Hands-on Activity

Create a webpage that includes:

- Your full name
- Your photograph (optional)
- Your school
- Your department
- Your hobbies
- Your favorite programming language
- Your career goal

Save the file as:

profile.html

Open it using Live Server.

Key Terms

Term	Meaning
HTML	HyperText Markup Language
Tag	A keyword enclosed in angle brackets used to define HTML elements
Element	A complete HTML component consisting of tags and content
Attribute	Extra information added to an HTML element
Browser	Software used to display web pages
Live Server	A VS Code extension for automatically refreshing webpages

Chapter Summary

In this chapter, you learned:

- The meaning of HTML.
- Why HTML is important.
- The difference between a markup language and a programming language.
- The structure of an HTML document.
- HTML tags, elements, and attributes.
- How to create and display your first webpage.
- Best practices for writing clean HTML code.

You are now ready to begin building webpages using more advanced HTML elements.

Review Questions

1. What does HTML stand for?
2. Why is HTML called a markup language?
3. Is HTML a programming language? Explain your answer.

4. What is the purpose of the <!DOCTYPE html> declaration?
5. Explain the purpose of the <head> and <body> elements.
6. What is an HTML tag?
7. What is an HTML attribute?
8. State the difference between an HTML element and an HTML tag.
9. Why is index.html commonly used as the homepage of a website?
10. Mention five best practices when writing HTML code.

Practical Assignment

Project: My Personal Webpage

Create a webpage named **index.html** that contains:

- A browser title.
- Your full name as the main heading.
- Three paragraphs introducing yourself.
- A horizontal line.
- A section titled **"My Career Goals"**.
- A paragraph describing your future career.
- A comment at the top of the file containing your name and the date.
- Proper indentation and formatting.

Submission Requirements:

- Submit the index.html file.
- Include a screenshot of the webpage displayed in your browser.
- Ensure your code is neatly formatted and follows the best practices discussed in this chapter.

CHAPTER 7
HTML TEXT FORMATTING AND SEMANTIC ELEMENTS

Learning Objectives

At the end of this chapter, students should be able to:

- Differentiate between physical and semantic HTML formatting tags.
- Format text using various HTML elements.
- Explain the purpose of semantic HTML elements.
- Improve webpage readability using proper text formatting.
- Write clean, meaningful, and accessible HTML code.
- Apply semantic HTML to enhance Search Engine Optimization (SEO) and accessibility.

Introduction

A webpage is more than just plain text. To make information clear and engaging, web developers use different formatting techniques such as **bold**, **italic**, **underlined**, **highlighted**, and **quoted** text.

HTML provides special elements for formatting text. Some of these elements simply change the appearance of the text, while others also provide meaning to browsers, search engines, and assistive technologies. These meaningful elements are known as **semantic elements**.

Understanding the difference between appearance and meaning is essential for creating professional, accessible, and SEO-friendly websites.

7.1 Physical vs. Semantic Formatting

HTML formatting elements fall into two categories:

Physical Formatting

These elements only affect how the text looks.

Examples:

- <b>
- <i>
- <u>

Semantic Formatting

These elements provide meaning to the content in addition to styling.

Examples:

-
-
- <mark>
- <abbr>

Modern web development encourages the use of **semantic elements** whenever possible.

7.2 Bold Text (<b>)

The <b> element displays text in **bold** without adding any special meaning.

Example

```
<p>This is <b>important</b> text.</p>
```

Output:

This is **important** text.

7.3 Strong Importance ()

The element indicates that the text is very important.

Browsers usually display it in **bold**, but it also carries semantic meaning.

Example

```
<p>Please <strong>save your work</strong> before closing the program.</p>
```

Best Practice

Use:

instead of

<b>

when emphasizing important information.

7.4 Italic Text (<i>)

The <i> element displays text in italics.

Example

```
<p>The word <i>Bonjour</i> means "Hello" in French.</p>
```

7.5 Emphasized Text ()

The element adds emphasis to text.

Browsers usually display it in italics, but it also provides semantic meaning.

Example

```
<p>You <em>must</em> complete this assignment.</p>
```

Best Practice

Use:

instead of

<i>

when the emphasis changes the meaning of the sentence.

7.6 Underlined Text (<u>)

The <u> element underlines text.

Example

```
<p><u>Web Development</u></p>
```

Note: Avoid underlining regular text on webpages because users often associate underlined text with hyperlinks.

7.7 Marked Text (<mark>)

The <mark> element highlights text.

Example

```
<p>HTML is the <mark>foundation</mark> of every website.</p>
```

7.8 Small Text (<small>)

Displays smaller text.

Example

```
<p><small>Terms and Conditions apply.</small></p>
```

7.9 Deleted Text (<del>)

Represents deleted content.

Example

```
<p>Old Price: <del>$100</del></p>
```

7.10 Inserted Text (<ins>)

Represents newly inserted content.

Example

```
<p>New Price: <ins>$80</ins></p>
```

7.11 Superscript (<sup>)

Displays text slightly above the normal line.

Common uses:

- Mathematical expressions
- Footnotes

- Exponents

Example

```
<p>x<sup>2</sup> + y<sup>2</sup></p>
```

Output:

x² + y²

7.12 Subscript (<sub>)

Displays text below the normal line.

Common uses:

- Chemical formulas
- Mathematical expressions

Example

```
<p>H<sub>2</sub>O</p>
```

Output:

H_2O

7.13 Quotations

Short Quotations (<q>)

```
<p>Albert Einstein once said,
<q>Imagination is more important than knowledge.</q>
</p>
```

Block Quotations (<blockquote>)

Used for longer quotations.

```
<blockquote>
Education is the passport to the future, for tomorrow belongs to
those who prepare for it today.
</blockquote>
```

7.14 Abbreviations (<abbr>)

Used to define abbreviations.

Example

```
<p>The <abbr title="World Health Organization">WHO</abbr> works globally.</p>
```

When users hover over **WHO**, the full meaning appears.

7.15 Code (<code>)

Displays programming code.

Example

```
<code>
print("Hello World")
</code>
```

7.16 Keyboard Input (<kbd>)

Represents keyboard shortcuts.

Example

```
Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save your work.
```

7.17 Sample Output (<samp>)

Displays output from a computer program.

Example

```
<samp>
Program executed successfully.
</samp>
```

7.18 Variables (<var>)

Represents variables in mathematical or programming expressions.

Example

<var>x</var> = 20

7.19 HTML Comments

Comments help explain code and are ignored by browsers.

Example

<!-- Navigation Menu Starts Here -->

Comments improve readability and teamwork.

7.20 Semantic HTML

Semantic HTML uses elements that describe the purpose of the content instead of just its appearance.

Examples include:

-
-
- <header>
- <nav>
- <section>
- <article>
- <footer>

Benefits include:

- Better accessibility
- Improved SEO
- Easier code maintenance
- Improved readability

Real-World Example

Consider an online newspaper article.

Instead of writing:

Breaking News

A professional developer writes:

Breaking News

Although both appear bold, tells browsers and assistive technologies that the content is important.

Worked Example

```
<!DOCTYPE html>
<html>
<head>
    <title>HTML Formatting</title>
</head>
<body>
<h1>HTML Formatting Example</h1>
<p><strong>Welcome</strong> to the <em>KJV Tech Web Development Masterclass</em>.</p>
<p>Today's topic is <mark>HTML Formatting Elements</mark>.</p>
<p>Water formula: H<sub>2</sub>O</p>
<p>Area formula: m<sup>2</sup></p>
<p>Press <kbd>Ctrl</kbd> + <kbd>S</kbd> to save.</p>
<p><abbr title="HyperText Markup Language">HTML</abbr> is the language of the web.</p>
</body>
</html>
```

Best Practices

- Use instead of when indicating importance.
- Use instead of <i> for emphasis.

- Keep formatting consistent throughout your webpages.
- Avoid excessive formatting, as it can reduce readability.
- Use semantic elements to improve accessibility and SEO.
- Comment your code where necessary.

Hands-on Activity

Create a webpage titled **"HTML Formatting Practice"**.

Include examples of:

- Bold text
- Strong text
- Italic text
- Emphasized text
- Underlined text
- Highlighted text
- Small text
- Deleted text
- Inserted text
- Superscript
- Subscript
- Abbreviation
- Quotation
- Code
- Keyboard shortcut

Open the page using **Live Server** and observe how each element is displayed.

Key Terms

Term	Meaning
Physical Formatting	Changes the appearance of text without adding meaning
Semantic Formatting	Adds meaning to content while also affecting its presentation
Accessibility	Designing web content so it can be used by everyone, including people with disabilities
SEO	Search Engine Optimization; practices that help webpages rank better in search engines
HTML Comment	A note in the code that is not displayed in the browser

Chapter Summary

In this chapter, you learned how to format text using HTML elements and the importance of using semantic elements over purely visual ones. You explored tags for emphasis, quotations, abbreviations, code, and keyboard input, and discovered how semantic HTML improves accessibility, maintainability, and search engine optimization.

Review Questions

1. What is the difference between physical and semantic formatting?
2. Explain the difference between <b> and .
3. When should you use instead of <i>?
4. What is the purpose of the <mark> element?
5. Explain the use of <sup> and <sub> with examples.
6. What is the purpose of the <abbr> element?

7. How does semantic HTML improve accessibility?
8. Why are HTML comments useful?
9. Write one example each of <code> and <kbd>.
10. List five semantic HTML elements.

Practical Assignment

Project: Student Profile Webpage

Create a webpage named **formatting.html** that contains:

- A main heading introducing yourself.
- A paragraph using and .
- An abbreviation for your school or department using <abbr>.
- A favorite quotation using <blockquote> or <q>.
- A programming code example using <code>.
- A keyboard shortcut using <kbd>.
- At least one example each of <mark>, <sup>, and <sub>.
- Well-formatted and properly indented HTML code.

Save the file, open it with **Live Server**, and submit both the HTML file and a screenshot of the webpage.

CHAPTER 8
HTML LISTS, LINKS, AND IMAGES

Learning Objectives

At the end of this chapter, students should be able to:

- Explain the importance of lists in web development.
- Create ordered, unordered, and description lists.
- Create internal, external, email, telephone, and anchor links.
- Insert images into a webpage.
- Apply image attributes correctly.
- Organize web-page navigation using hyperlinks.
- Follow best practices for using links and images.

Introduction

Imagine visiting a website where all the information is written in one long paragraph with no images or navigation. It would be difficult to read and almost impossible to navigate. Lists help organize information. Links connect webpages together. Images make web-pages attractive and informative. Together, these elements form the foundation of almost every modern website.

8.1 HTML Lists

Lists help present information in an organized manner.

HTML provides three main types of lists:

- Ordered Lists
- Unordered Lists
- Description Lists

8.2 Ordered Lists (<ol>)

An ordered list displays items in a numbered sequence.

Syntax

```
<ol>
```

```
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
</ol>
```

Output

1. HTML
2. CSS
3. JavaScript

Changing the Numbering Style

Alphabetical

```
<ol type="A">
    <li>Home</li>
    <li>About</li>
    <li>Contact</li>
</ol>
```

Lowercase Letters

```
<ol type="a">
```

Roman Numerals

```
<ol type="I">
```

or

```
<ol type="i">
```

8.3 Unordered Lists (<ul>)

An unordered list displays bullet points instead of numbers.

Example

```
<ul>
    <li>Laptop</li>
    <li>Keyboard</li>
    <li>Mouse</li>
</ul>
```

Output

- Laptop
- Keyboard
- Mouse

Bullet Styles

```
<ul type="circle">
<ul type="square">
<ul type="disc">
```

> **Note:** Modern HTML favors using CSS to style bullets instead of the type attribute.

8.4 Description Lists (<dl>)

A description list displays terms with their descriptions.

```
<dl>

<dt>HTML</dt>

<dd>HyperText Markup Language</dd>

<dt>CSS</dt>

<dd>Cascading Style Sheets</dd>

</dl>
```

8.5 Nested Lists

Lists can be placed inside other lists.

```
<ul>

<li>Programming

    <ul>

        <li>HTML</li>

        <li>CSS</li>

        <li>JavaScript</li>
```

```
    </ul>
</li>
</ul>
```

Nested lists are commonly used for menus and categories.

8.6 Hyperlinks

A hyperlink allows users to move from one webpage to another.

The HTML tag used is:

<a>

The href attribute specifies the destination.

8.7 External Links

An external link opens another website.

```
<a href="https://www.google.com">
Visit Google
</a>
```

Opening in a New Tab

```
<a href="https://www.google.com" target="_blank">
Visit Google
</a>
```

8.8 Internal Links

Internal links connect pages within the same website.

Example folder:

Website/

index.html

about.html

contact.html

Example

```
<a href="about.html">
About Us
</a>
```

8.9 Email Links

Email links automatically open the user's email application.

```
<a href="mailto:info@kjvtech.com">
Email Us
</a>
```

8.10 Telephone Links

Telephone links allow users to call directly on supported devices.

```
<a href="tel:+2348012345678">
Call Us
</a>
```

8.11 Anchor Links

Anchor links move users to another section of the same webpage.

Step 1

Create an ID.

```
<h2 id="contact">
Contact Us
</h2>
```

Step 2

Create the link.

```
<a href="#contact">
Go to Contact Section
</a>
```

8.12 Images in HTML

Images make websites more attractive and engaging.

The HTML image tag is:

Unlike most HTML elements, <img> does not have a closing tag.

8.13 Image Syntax

<img src="images/logo.png" alt="KJV Tech Logo">

8.14 Image Attributes

src

Specifies the image location.

Example:

src="student.jpg"

alt

Provides alternative text if the image cannot be displayed.

Example:

alt="Student Picture"

The alt attribute also improves accessibility for screen readers.

width

Specifies image width.

width="300"

height

Specifies image height.

height="200"

title

Displays additional information when users hover over the image.

title="KJV Tech Logo"

8.15 Image File Formats

Common image formats include:

Format	Best Use
JPG/JPEG	Photographs
PNG	Logos and images with transparent backgrounds
GIF	Simple animations
SVG	Icons and vector graphics
WebP	Modern compressed web images

8.16 Organizing Images

Professional developers store images in a dedicated folder.

Example:

MyWebsite/

index.html

images/

logo.png

student.jpg

background.jpg

8.17 Best Practices for Images

- Always use the alt attribute.
- Resize large images before uploading.
- Use descriptive file names.
- Keep images inside an images folder.
- Use optimized formats like WebP when appropriate.
- Avoid using excessively large image files.

8.18 Combining Lists, Links, and Images

Example webpage:

```
<!DOCTYPE html>
<html>
<head>
<title>My Website</title>
</head>
<body>
<h1>Welcome to KJV Tech</h1>
<img src="images/logo.png" alt="KJV Tech Logo" width="150">
<h2>Navigation</h2>
<ul>
<li><a href="index.html">Home</a></li>
<li><a href="about.html">About</a></li>
<li><a href="contact.html">Contact</a></li>
</ul>
<h2>Courses Offered</h2>
<ol>
<li>HTML</li>
<li>CSS</li>
<li>JavaScript</li>
<li>PHP</li>
</ol>
<p>
Visit our
<a href="https://www.kjvtech.vercel.app" target="_blank">
official website
</a>.
</p>

</body>

</html>
```

Real-World Application

Almost every website uses these HTML elements:

- **E-commerce websites** use lists to display product categories.
- **University websites** use links for admissions, departments, and student portals.
- **News websites** use images to accompany articles.
- **Business websites** use navigation menus built with lists and links.

Best Practices

- Use meaningful link text instead of "Click Here."
- Always provide an alt attribute for images.
- Organize images in a dedicated folder.
- Check that all links work before publishing your website.
- Keep navigation menus simple and easy to use.
- Optimize images to improve page loading speed.

Hands-on Activity

Create a webpage containing:

- A website logo.
- Your photograph.
- An unordered navigation menu with links to:
    - Home
    - About
    - Services
    - Contact
- An ordered list of your favorite programming languages.
- A description list explaining HTML, CSS, and JavaScript.
- An external link to your favorite educational website.
- An email link.
- A telephone link.

Open the webpage using **Live Server** and test every link.

Key Terms

Term	Meaning
Ordered List	A numbered list created with
Unordered List	A bulleted list created with
Description List	A list of terms and descriptions using <dl>, <dt>, and <dd>
Hyperlink	A clickable connection to another resource
Anchor Link	A link to a section within the same webpage
Image	A visual element inserted using the <img> tag
alt Attribute	Alternative text describing an image

Chapter Summary

In this chapter, you learned how to organize webpage content using lists, connect webpages with different types of hyperlinks, and enhance web pages with images. You also explored image attributes, file organization, and best practices for accessibility and performance. These skills are essential for building well-structured and user-friendly websites.

Review Questions

1. What is the difference between an ordered list and an unordered list?
2. Explain the purpose of a description list.
3. What is the function of the <a> tag?
4. Differentiate between internal and external links.
5. What is the purpose of the target="_blank" attribute?
6. Explain the purpose of the alt attribute in an image.
7. Name four common image file formats and their typical uses.
8. What is an anchor link, and when would you use one?
9. Why should images be stored in a dedicated folder?
10. List five best practices for using hyperlinks and images on a website.

Practical Assignment

Project: Multi-Page Personal Website

Create a website with the following pages:

- **Home** (index.html)
- **About** (about.html)
- **Gallery** (gallery.html)
- **Contact** (contact.html)

Requirements

- A navigation menu using an unordered list on every page.
- Internal links connecting all four pages.
- At least one external link that opens in a new tab.
- An email link and a telephone link on the Contact page.
- A minimum of three images with appropriate alt text.
- An ordered list of your favorite web development tools.
- A description list defining HTML, CSS, and JavaScript.
- Organize all images in an images folder and ensure the folder structure is clean.

Submission Requirements:

- Submit the complete project folder.
- Verify that all links work correctly.
- Ensure images display properly and all HTML code is neatly formatted.

CHAPTER 9
HTML TABLES

Learning Objectives

At the end of this chapter, students should be able to:

- Explain the purpose of HTML tables.
- Create tables using rows and columns.
- Use table headers, captions, and data cells.
- Merge rows and columns using rowspan and colspan.
- Organize tables with <thead>, <tbody>, and <tfoot>.
- Build professional tables for displaying structured data.
- Apply best practices for designing accessible and readable tables.

Introduction

Tables are one of the most useful HTML elements for displaying **structured information**. Whenever you need to present data in rows and columns—such as student results, employee records, product prices, or class timetables—HTML tables provide a clear and organized way to do so.

A well-designed table makes information easier to understand, compare, and analyze. In this chapter, you will learn how to create simple and advanced tables using HTML5.

Important: Tables should only be used for displaying tabular data, not for creating webpage layouts. Modern websites use CSS layout techniques such as Flexbox and Grid for page design.

9.1 What is an HTML Table?

An HTML table is a structure used to organize data into:

- Rows
- Columns
- Cells

A table is created using the <table> element.

9.2 Basic Table Structure

The basic HTML tags used in tables are:

Tag	Purpose
<table>	Creates the table
<tr>	Defines a table row
<th>	Defines a table header cell
<td>	Defines a table data cell
<caption>	Adds a title to the table

Example

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Table</title>
</head>
<body>

<table border="1">

<tr>
    <th>Name</th>
    <th>Department</th>
    <th>Level</th>
</tr>

<tr>
    <td>John</td>
    <td>Computer Science</td>
    <td>200</td>
</tr>
```

```
<tr>
    <td>Mary</td>
    <td>Software Engineering</td>
    <td>300</td>
</tr>

</table>

</body>
</html>
```

9.3 Table Caption

A caption gives the table a title.

Example

```
<table border="1">

<caption>
Student Information
</caption>

...
</table>
```

The caption appears above the table by default.

9.4 Table Headers (<th>)

Header cells describe the information contained in each column.

Example:

```
<tr>

<th>Name</th>

<th>Age</th>

<th>Course</th>

</tr>
```

Browsers display header cells in **bold** and **center-aligned** by default.

9.5 Table Data (<td>)

Table data cells contain the actual information.

Example:

```
<tr>
<td>Joshua</td>
<td>20</td>
<td>Web Development</td>
</tr>
```

9.6 Table Rows (<tr>)

Each <tr> element represents one row in the table.

Example:

```
<tr>
<td>Monday</td>
<td>HTML Class</td>
</tr>
```

9.7 Adding Borders

Older HTML examples use the border attribute:

```
<table border="1">
```

However, modern web development uses **CSS** for borders.

Example:

```
table{
    border:1px solid black;
}
```

You will learn CSS styling in Module 3.

9.8 Cell Padding and Spacing

Older HTML used:

cellpadding="10"

cellspacing="5"

Today, CSS is recommended for spacing.

9.9 Merging Columns (colspan)

Sometimes one cell should span multiple columns.

Example

```
<table border="1">

<tr>

<th colspan="3">
Student Result
</th>

</tr>

<tr>

<th>Name</th>

<th>Course</th>

<th>Score</th>

</tr>

</table>
```

The title stretches across three columns.

9.10 Merging Rows (rowspan)

A cell can also extend across multiple rows.

Example

```
<table border="1">

<tr>

<th rowspan="2">
Department
</th>

<th>Student</th>

</tr>

<tr>

<td>John</td>

</tr>

</table>
```

9.11 Table Sections

Professional HTML tables are divided into three sections.

<thead>

Contains the table headings.

Example:

```
<thead>

<tr>

<th>Name</th>

<th>Score</th>

</tr>

</thead>
```

<tbody>

Contains the main table data.

Example:

```
<tbody>

<tr>

<td>David</td>

<td>90</td>

</tr>

</tbody>
```

<tfoot>

Contains summary information.

Example:

```
<tfoot>

<tr>

<td>Total Students</td>

<td>30</td>

</tr>

</tfoot>
```

9.12 Complete Professional Table

```
<!DOCTYPE html>
<html>

<head>
<title>Student Scores</title>
```

```
</head>
<body>
<table border="1">
<caption>Student Examination Results</caption>
<thead>
<tr>
<th>Name</th>
<th>Department</th>
<th>Score</th>
</tr>
</thead>
<tbody>
<tr>
<td>Joshua</td>
<td>Computer Science</td>
<td>85</td>
</tr>
<tr>
<td>Grace</td>
<td>Information Technology</td>
<td>91</td>
</tr>
<tr>
```

```
<td>Daniel</td>

<td>Cyber Security</td>

<td>78</td>

</tr>

</tbody>

<tfoot>

<tr>

<td colspan="2">
Average Score
</td>

<td>84.7</td>

</tr>

</tfoot>

</table>

</body>

</html>
```

9.13 Real-World Applications of Tables

HTML tables are commonly used for:

- Student examination results
- School timetables
- Product price lists
- Employee records
- Financial reports
- Sales reports
- Event schedules
- Inventory management

9.14 Accessibility in Tables

To make tables accessible:

- Use <th> for headings instead of <td>.
- Add a <caption> to describe the table.
- Organize content with <thead>, <tbody>, and <tfoot>.
- Keep tables simple and easy to understand.
- Avoid using tables for page layouts.

These practices help screen readers interpret tables correctly.

Best Practices

- Use tables only for tabular data.
- Include meaningful column headings.
- Add captions to describe tables.
- Use CSS instead of HTML attributes for styling.
- Keep tables organized and readable.
- Test tables on different screen sizes.

Worked Example

Class Timetable

```
<table border="1">

<caption>Weekly Class Timetable</caption>

<tr>

<th>Day</th>

<th>9:00–11:00 AM</th>

<th>11:30–1:30 PM</th>

</tr>

<tr>
```

```
<td>Monday</td>

<td>HTML</td>

<td>CSS</td>

</tr>

<tr>

<td>Wednesday</td>

<td>JavaScript</td>

<td>PHP</td>

</tr>

<tr>

<td>Friday</td>

<td>Database</td>

<td>Project Work</td>

</tr>

</table>
```

Hands-on Activity

Create a table containing:

- Student Name
- Department
- Level
- Email Address
- Phone Number

Requirements:

- Add a caption.
- Include at least five students.

- Use <thead>, <tbody>, and <tfoot>.
- Add a summary row showing the total number of students.

Open the page using **Live Server** and verify that the table is displayed correctly.

Key Terms

Term	Meaning
Table	A structure for displaying data in rows and columns
Row	A horizontal line of cells in a table
Column	A vertical set of cells in a table
Cell	An individual data box within a table
<th>	Table header cell
<td>	Table data cell
<caption>	Title of a table
rowspan	Merges multiple rows into one cell
colspan	Merges multiple columns into one cell
<thead>	Groups table header content
<tbody>	Groups the main table content
<tfoot>	Groups summary or footer content

Chapter Summary

In this chapter, you learned how to create and organize HTML tables using rows, columns, headers, captions, and data cells. You also explored advanced features such as merging cells with rowspan and colspan, structuring tables with <thead>, <tbody>, and <tfoot>, and applying accessibility and coding best practices. These skills

are essential for presenting structured information clearly and professionally.

Review Questions

1. What is the purpose of an HTML table?
2. Explain the functions of <table>, <tr>, <th>, and <td>.
3. What is the purpose of the <caption> element?
4. Differentiate between <th> and <td>.
5. Explain the use of rowspan with an example.
6. Explain the use of colspan with an example.
7. What are the roles of <thead>, <tbody>, and <tfoot>?
8. Why should tables not be used for webpage layouts?
9. Mention five real-world applications of HTML tables.
10. List five best practices for creating HTML tables.

Practical Assignment

Project: Student Result Management Table

Create a webpage named results.html containing a professional examination result table.

Requirements

- Add a table caption titled **"Student Examination Results"**.
- Include the following columns:
    - Student ID
    - Student Name
    - Course
    - Continuous Assessment (CA)
    - Examination Score
    - Total Score
    - Grade
- Enter data for at least **10 students**.
- Use <thead>, <tbody>, and <tfoot>.
- Merge cells where appropriate using colspan or rowspan.

- Add a footer row showing the class average or total number of students.
- Ensure the HTML code is properly indented and well-commented.

Submission Requirements:

- Submit the results.html file.
- Include a screenshot of the table displayed in your browser.
- Verify that the table is clear, readable, and accurately structured.

KJV TECH

CHAPTER 10

HTML FORMS

Learning Objectives

At the end of this chapter, students should be able to:

- Explain the purpose of HTML forms.
- Create forms to collect user information.
- Use different HTML form controls such as text fields, radio buttons, checkboxes, dropdown menus, and text areas.
- Apply HTML5 input types for better user experience.
- Use form validation attributes to ensure accurate data entry.
- Build professional registration and contact forms.
- Follow best practices for designing user-friendly forms.

Introduction

Imagine trying to sign up for a social media account, purchase a product online, or apply for admission to a university. In each case, you are required to enter information such as your name, email address, phone number, or password. This information is collected through **HTML forms**.

Forms are one of the most important parts of web development because they allow users to interact with websites by sending data to a server. In this chapter, you will learn how to build forms using HTML5 and understand the various input controls available.

10.1 What is an HTML Form?

An HTML form is a section of a webpage that collects user input.

Forms can be used for:

- User registration
- Login pages
- Contact forms
- Job applications

- Online surveys
- Student admission forms
- Online shopping checkout

Forms are created using the <form> element.

10.2 The <form> Element

The basic structure of a form is:

```
<form action="submit.php" method="post">

</form>
```

Attributes

Attribute	Purpose
action	Specifies where the form data is sent
method	Specifies how the data is sent (GET or POST)

10.3 GET vs POST Methods

GET Method

- Appends form data to the URL.
- Suitable for search forms.
- Not recommended for sensitive information.

Example:

```
https://example.com/search?name=John
```

POST Method

- Sends data in the request body.
- More secure than GET.
- Used for login, registration, and payment forms.

10.4 Labels (<label>)

Labels describe form controls and improve accessibility.

Example:

```
<label for="fullname">Full Name</label>
<input type="text" id="fullname" name="fullname">
```

10.5 Text Input

Used to collect short text.

Example:

```
<input type="text" name="fullname">
```

10.6 Password Field

Masks characters as the user types.

Example:

```
<input type="password" name="password">
```

10.7 Email Field

Accepts only valid email addresses.

Example:

```
<input type="email" name="email">
```

10.8 Number Field

Accepts only numeric values.

Example:

```
<input type="number" name="age">
```

10.9 Date Picker

Allows users to select a date.

Example:

```
<input type="date" name="dob">
```

10.10 Telephone Field

Used for telephone numbers.

Example:

```
<input type="tel" name="phone">
```

10.11 URL Field

Accepts website addresses.

Example:

```
<input type="url" name="website">
```

10.12 Radio Buttons

Allow users to choose only one option.

Example:

```
<p>Gender</p>

<input type="radio" id="male" name="gender" value="Male">
<label for="male">Male</label>

<input type="radio" id="female" name="gender" value="Female">
<label for="female">Female</label>
```

10.13 Checkboxes

Allow users to select multiple options.

Example:

```
<p>Programming Languages</p>

<input type="checkbox" id="html" name="language" value="HTML">
<label for="html">HTML</label>

<input type="checkbox" id="css" name="language" value="CSS">
<label for="css">CSS</label>

<input type="checkbox" id="js" name="language" value="JavaScript">
<label for="js">JavaScript</label>
```

10.14 Dropdown List (<select>)

Used when selecting one option from a list.

Example:

```
<label for="department">Department</label>

<select id="department" name="department">

<option>Computer Science</option>

<option>Software Engineering</option>

<option>Cyber Security</option>

<option>Information Technology</option>

</select>
```

10.15 Text Area

Used for long text such as comments or messages.

Example:

```
<textarea rows="5" cols="40"></textarea>
```

10.16 File Upload

Allows users to upload files.

Example:

```
<input type="file" name="passport">
```

10.17 Buttons

Submit Button

```
<input type="submit" value="Submit">
```

or

```
<button type="submit">Submit</button>
```

Reset Button

```
<input type="reset" value="Reset">
```

Clears all entered information.

Button Element

```
<button type="button">Click Me</button>
```

Often used with JavaScript.

10.18 HTML5 Form Validation

HTML5 provides built-in validation.

Required Field

```
<input type="text" required>
```

Placeholder

Displays helpful text inside a field.

```
<input type="text" placeholder="Enter your full name">
```

Minimum and Maximum

```
<input type="number" min="16" max="60">
```

Minimum Length

```
<input type="password" minlength="8">
```

Maximum Length

```
<input type="text" maxlength="30">
```

Pattern Validation

Restricts input using regular expressions.

Example:

```
<input type="text" pattern="[A-Za-z ]+">
```

Only letters and spaces are accepted.

10.19 Complete Registration Form Example

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration Form</title>
</head>
<body>

<h2>Student Registration Form</h2>

<form action="#" method="post">
```

```
<label for="fullname">Full Name</label><br>
<input type="text" id="fullname" name="fullname"
placeholder="Enter your full name" required><br><br>

<label for="email">Email Address</label><br>
<input type="email" id="email" name="email" required><br><br>

<label for="password">Password</label><br>
<input type="password" id="password" name="password" minlength="8"
required><br><br>

<label for="age">Age</label><br>
<input type="number" id="age" name="age" min="16"
max="100"><br><br>

<label>Gender</label><br>

<input type="radio" id="male" name="gender" value="Male">
<label for="male">Male</label>

<input type="radio" id="female" name="gender" value="Female">
<label for="female">Female</label>

<br><br>

<label for="department">Department</label><br>

<select id="department" name="department">

<option>Computer Science</option>

<option>Cyber Security</option>

<option>Software Engineering</option>

<option>Information Technology</option>

</select>

<br><br>

<label>Skills</label><br>

<input type="checkbox"> HTML
```

```
<input type="checkbox"> CSS

<input type="checkbox"> JavaScript

<input type="checkbox"> PHP

<br><br>

<label for="message">About Yourself</label><br>

<textarea id="message" rows="5" cols="40"></textarea>

<br><br>

<label for="photo">Passport Photograph</label><br>

<input type="file" id="photo">

<br><br>

<button type="submit">Register</button>

<button type="reset">Reset</button>

</form>

</body>
</html>
```

10.20 Best Practices for HTML Forms

- Always use <label> for every form control.
- Group related fields together.
- Use appropriate input types (email, date, tel, etc.).
- Validate user input using HTML5 attributes.
- Keep forms short and easy to complete.
- Provide meaningful placeholder text.
- Clearly indicate required fields.
- Test forms on desktop and mobile devices.

Real-World Applications

HTML forms are used in:

- User login pages
- Online registration systems
- University admission portals
- Banking applications
- E-commerce checkout pages
- Hotel booking systems
- Online surveys
- Customer feedback forms

Hands-on Activity

Create a **Student Admission Form** that includes:

- Full Name
- Email Address
- Password
- Age
- Date of Birth
- Gender
- Department
- State of Origin
- Phone Number
- Skills (checkboxes)
- Passport Photograph upload
- Personal Statement
- Submit and Reset buttons

Apply appropriate HTML5 validation such as required, min, max, maxlength, and pattern where necessary.

Key Terms

Term	Meaning
Form	A section of a webpage used to collect user input
Input	A form control for entering data

Term	Meaning
Label	Describes a form control
Placeholder	Hint text displayed inside an input field
Validation	Checking whether user input meets specified rules
Radio Button	Allows selection of one option from a group
Checkbox	Allows selection of multiple options
Dropdown	A list of selectable options
Textarea	A multi-line text input
Submit Button	Sends form data to the server

Chapter Summary

In this chapter, you learned how to create HTML forms and use a variety of input controls to collect user information. You explored text fields, password fields, radio buttons, checkboxes, dropdown lists, text areas, file uploads, and buttons. You also learned how to use HTML5 validation features to improve data quality and user experience. These skills are fundamental to creating interactive websites and web applications.

Review Questions

1. What is an HTML form?
2. Explain the purpose of the action and method attributes.
3. Differentiate between the GET and POST methods.
4. Why are <label> elements important?
5. What is the difference between radio buttons and checkboxes?
6. Explain the purpose of the <select> element.
7. What is the function of the required attribute?
8. How does the pattern attribute improve form validation?
9. List five HTML5 input types and their uses.
10. State five best practices for designing HTML forms.

Practical Assignment

Project: Online Student Registration System

Create a webpage named registration.html that contains a complete student registration form.

Requirements

Include the following fields:

- Student ID
- Full Name
- Email Address
- Password
- Confirm Password
- Date of Birth
- Age
- Gender
- Phone Number
- Department (dropdown)
- Level (dropdown)
- State of Origin
- Skills (checkboxes)
- Passport Photograph (file upload)
- Personal Statement (<textarea>)
- Terms and Conditions (checkbox)
- Submit and Reset buttons

Validation Requirements

- All mandatory fields must use the required attribute.
- Password must have a minimum length of 8 characters.
- Age must accept values between **16** and **100**.
- Email must use the email input type.
- Phone number should accept only valid numeric input using an appropriate pattern.

Submission Requirements

- Save the file as registration.html.
- Test the form in your browser using **Live Server**.
- Capture a screenshot showing the completed form.
- Ensure the HTML code is properly indented and commented.

CHAPTER 11

HTML5 MULTIMEDIA

Learning Objectives

At the end of this chapter, students should be able to:

- Explain the importance of multimedia in web development
- Embed images, audio, and video into webpages.
- Use HTML5 audio and video elements with appropriate attributes.
- Embed external content using <iframe>.
- Add YouTube videos and Google Maps to webpages.
- Explain the difference between SVG and Canvas graphics.
- Apply accessibility and performance best practices when using multimedia.

Introduction

Modern websites are not limited to text and hyperlinks. They also contain photographs, music, videos, animations, maps, and interactive graphics.

HTML5 provides built-in elements that allow developers to integrate multimedia content without relying on additional browser plugins.

In this chapter, you will learn how to enrich webpages with multimedia and create engaging user experiences using HTML.

11.1 What Is Multimedia in Web Development?

Multimedia refers to the combination of different forms of content on a webpage, including:

- Text
- Images
- Audio
- Video
- Animations
- Interactive graphics

Multimedia is used in educational platforms, news websites, online stores, entertainment websites, and business presentations.

11.2 Embedding Images

Images are inserted using the <img> element.

Example

```
<img src="images/web-development.jpg"
     alt="A computer displaying a website"
     width="500">
```

Important attributes:

Attribute	Purpose
src	Specifies the image location
alt	Provides alternative text
width	Sets the image width
height	Sets the image height

For decorative images, use an empty alt attribute (alt=""). For meaningful images, provide descriptive alternative text.

11.3 HTML5 Audio

The <audio> element allows developers to embed sound files directly into webpages.

Basic Syntax

```
<audio controls>
    <source src="audio/music.mp3" type="audio/mpeg">
    Your browser does not support audio playback.
</audio>
```

Common Audio Attributes

Attribute	Purpose
controls	Displays playback controls
autoplay	Attempts to play audio automatically
loop	Repeats the audio
muted	Starts the audio muted
preload	Suggests how much audio to load in advance

Best practice: Provide playback controls and avoid unexpected autoplay with sound.

11.4 HTML5 Video

The <video> element allows developers to display videos without requiring external plugins.

Example

```
<video width="640" controls>
    <source src="videos/tutorial.mp4" type="video/mp4">
    Your browser does not support video playback.
</video>
```

Common Video Attributes

Attribute	Purpose
controls	Displays playback controls
width	Sets the video width
height	Sets the video height
autoplay	Attempts automatic playback
muted	Starts the video muted
loop	Repeats the video
poster	Displays an image before playback

Example with Poster

```
<video controls width="640"
       poster="images/video-cover.jpg">
    <source src="videos/tutorial.mp4" type="video/mp4">
</video>
```

11.5 Adding Subtitles and Captions

The <track> element provides subtitles, captions, and other timed text for video.

```
<video controls width="640">
    <source src="videos/lesson.mp4" type="video/mp4">

    <track src="captions/lesson-en.vtt"
           kind="captions"
```

```
            srclang="en"
            label="English"
            default>
</video>
```

A WebVTT caption file (.vtt) contains timed text that synchronizes with the video.

Example:

```
WEBVTT

00:00:01.000 --> 00:00:04.000
Welcome to the web development lesson.
```

Captions improve accessibility and help users understand videos when audio is unavailable.

11.6 Embedding YouTube Videos

YouTube videos can be embedded using an <iframe>.

```
<iframe
    width="560"
    height="315"
    src="https://www.youtube.com/embed/VIDEO_ID"
    title="Web Development Tutorial"
    allowfullscreen>
</iframe>
```

Replace VIDEO_ID with the actual YouTube video ID.

For example, a video URL such as:

```
https://www.youtube.com/watch?v=abc123XYZ
```

uses abc123XYZ as its video ID.

11.7 Embedding Google Maps

Google Maps can be embedded using an iframe.

Steps

- Open Google Maps.
- Search for the desired location.
- Select Share.
- Choose Embed a map.
- Copy the HTML code.
- Paste it into your webpage.

Example:

```
<iframe
    src="https://www.google.com/maps/embed?pb=YOUR_MAP_CODE"
    width="600"
    height="450"
    title="Training Venue Map"
    loading="lazy">
</iframe>
```

Replace YOUR_MAP_CODE with the embed URL provided by Google Maps.

11.8 The <iframe> Element

An iframe embeds another webpage or external resource inside the current webpage.

Example

```
<iframe
    src="https://example.com"
    width="600"
    height="400"
    title="Embedded Example Website">
</iframe>
```

Common uses include

- YouTube video
- Google Maps
- Online documents
- External web applications

Only embed content from trusted sources, and always provide a meaningful title attribute.

11.9 SVG Graphics

SVG stands for Scalable Vector Graphics.

SVG images are defined using XML-based markup and can be scaled without losing quality.

Example

```
<svg width="200" height="120"
     viewBox="0 0 200 120"
     xmlns="http://www.w3.org/2000/svg">

    <rect width="200" height="120" fill="lightblue"/>

    <circle cx="100" cy="60" r="40" fill="orange"/>

</svg>
```

This code creates a light-blue rectangle containing an orange circle.

SVG is useful for:

- Logos
- Icons
- Diagrams
- Charts
- Scalable illustrations

11.10 HTML Canvas

The <canvas> element provides a drawing area that JavaScript can use to create graphics and animations.

Example

```
<canvas id="myCanvas"
        width="300"
        height="150">
    Your browser does not support canvas.
</canvas>

<script>
    const canvas = document.getElementById("myCanvas");
    const ctx = canvas.getContext("2d");
```

```
    ctx.fillStyle = "blue";
    ctx.fillRect(50, 30, 200, 90);
</script>
```

This example draws a blue rectangle inside the canvas.

SVG vs. Canvas

SVG	Canvas
Vector-based	Pixel-based
Elements remain individually accessible	Drawing is rendered as pixels
Suitable for logos and diagrams	Suitable for games and dynamic graphics
Scales without losing quality	May lose quality when enlarged

11.11 Complete Multimedia Webpage

Create a file named multimedia.html and add the following code:

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>KJV Tech Multimedia</title>
</head>

<body>

    <header>
        <h1>Welcome to KJV Tech</h1>
        <p>Learn technology through practical lessons.</p>
    </header>

    <main>

        <section>
            <h2>Our Training</h2>

            <img src="images/training.jpg"
```

```
                alt="Students learning web development"
                width="400">
        </section>

        <section>
            <h2>Audio Lesson</h2>

            <audio controls>
                <source src="audio/lesson.mp3" type="audio/mpeg">
            </audio>
        </section>

        <section>
            <h2>Video Tutorial</h2>

            <video controls width="500">
                <source src="videos/tutorial.mp4"
type="video/mp4">
            </video>
        </section>

        <section>
            <h2>Watch on YouTube</h2>

            <iframe
                width="560"
                height="315"
                src="https://www.youtube.com/embed/VIDEO_ID"
                title="HTML Tutorial"
                allowfullscreen>
            </iframe>
        </section>

    </main>

    <footer>
        <p>&copy; 2026 KJV Tech</p>
    </footer>

</body>
</html>

Remember to replace the example file paths and VIDEO_ID with
actual resources.
```

11.12 Multimedia Best Practices

- Use descriptive alternative text for meaningful images
- Provide captions or subtitles for videos when possible.
- Include playback controls for audio and video.
- Optimize media files to reduce loading time
- Use responsive dimensions for different screen sizes.
- Avoid autoplay with sound.
- Provide fallback text for unsupported media.
- Use meaningful titles for embedded content.
- Embed external content only from trusted sources.

Hands-on Activity

Create a webpage titled "My Multimedia Learning Page" containing:

1. A heading and introductory paragraph.
2. Two images with appropriate alternative text.
3. An audio player containing an audio file.
4. A video player containing a video file.
5. An embedded YouTube tutorial.
6. An embedded Google Maps location.
7. An SVG illustration containing at least two shapes

Open the page using Live Server and test all multimedia elements.

Key Terms

Term	Meaning
Multimedia	A combination of different types of media
Audio	Sound content embedded in a webpage
Video	Moving visual content
iframe	An element used to embed external content
SVG	Scalable Vector Graphics
Canvas	A drawing area controlled through JavaScript
WebVTT	A format for timed text such as captions
Accessibility	Making content usable by people with different abilities

Chapter Summary

In this chapter, you learned how to embed images, audio, video, and external content into webpages. You explored YouTube and Google Maps embedding, SVG graphics, and the HTML Canvas element. You also learned how accessibility, media optimization, and responsive design contribute to a better user experience.

Review Questions

- What is multimedia in web development?
- Explain the purpose of the <audio> element.
- State five attributes of the <video> element.
- What is the purpose of the <track> element?
- How do you embed a YouTube video into a webpage?
- What is an iframe?
- Explain the difference between SVG and Canvas.
- Mention four uses of SVG graphics.
- Why should multimedia files be optimized?
- List five best practices for embedding multimedia.

Practical Assignment

Project: Educational Multimedia Website

Create a webpage named learning.html for a fictional educational platform.

Requirements

- Include a heading and introduction.
- Embed at least two images.
- Add an audio lesson with playback controls.
- Add a video lesson with playback controls.
- Embed a YouTube educational video.
- Embed a Google Maps location.
- Create an SVG graphic.
- Add captions to the video if a caption file is available.
- Organize media in separate images, audio, and videos folders.

Submission: Submit the complete project folder and a screenshot showing the webpage in a browser.

MODULE PROJECT: PERSONAL PORTFOLIO WEBSITE

Project Overview

In this project, you will build a personal portfolio website using HTML. The website will introduce you, present your skills and projects, and provide a way for visitors to contact you.

The project will help you apply the HTML concepts covered in Chapters 6–13.

Project Objectives

By completing this project, students should be able to:

- Create a complete HTML document.
- Use semantic HTML elements to organize content.
- Add navigation links, images, lists, and tables where appropriate.
- Create an accessible contact form.
- Apply HTML best practices.

Website Requirements

Your portfolio website must include the following sections:

- Header: Your name or portfolio title and a short tagline.
- Navigation: Links to the main sections of the page.
- About Me: A short introduction about yourself.
- Skills: A list of technical skills or subjects you are learning.
- Projects: At least two projects, each with a title, description, and link.
- Education: Your educational background.
- Contact: A form with fields for name, email, and message.
- Footer: Copyright information and optional social links.

Suggested File Structure

```
portfolio/
├── index.html
└── images/
    └── profile.jpg
```

Starter HTML

Create a file named index.html and begin with this structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <meta name="description" content="Personal portfolio of a
beginner web developer">
  <title>My Portfolio</title>
</head>
<body>

  <header>
    <h1>Your Name</h1>
    <p>Web Developer in Training</p>
  </header>

  <nav aria-label="Main navigation">
    <a href="#about">About</a>
    <a href="#skills">Skills</a>
    <a href="#projects">Projects</a>
    <a href="#education">Education</a>
    <a href="#contact">Contact</a>
  </nav>

  <main>
    <section id="about">
      <h2>About Me</h2>
      <p>Write a short introduction about yourself.</p>
      <img src="images/profile.jpg" alt="Portrait of Your Name">
    </section>

    <section id="skills">
      <h2>My Skills</h2>
      <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
      </ul>
    </section>

    <section id="projects">
```

```
        <h2>My Projects</h2>

        <article>
          <h3>Project One</h3>
          <p>Describe your first project.</p>
          <a href="#">View Project</a>
        </article>

        <article>
          <h3>Project Two</h3>
          <p>Describe your second project.</p>
          <a href="#">View Project</a>
        </article>
      </section>

      <section id="education">
        <h2>Education</h2>
        <p>Describe your educational background.</p>
      </section>

      <section id="contact">
        <h2>Contact Me</h2>

        <form action="#" method="post">
          <label for="name">Name:</label>
          <input type="text" id="name" name="name" required>

          <label for="email">Email:</label>
          <input type="email" id="email" name="email" required>

          <label for="message">Message:</label>
          <textarea id="message" name="message" rows="5"
required></textarea>

          <button type="submit">Send Message</button>
        </form>
      </section>
    </main>

    <footer>
      <p>&copy; 2026 Your Name. All rights reserved.</p>
    </footer>

</body>
</html>
```

Note: The contact form is a frontend demonstration. To send messages, it will need a backend or a form-processing service.

Assessment Checklist

Project completion

0/10

- [ ] Complete HTML document structure
- [ ] Semantic elements used appropriately
- [ ] Navigation links work
- [ ] About section includes an introduction
- [ ] Skills list contains at least three items
- [ ] At least two projects are presented
- [ ] Education section is included
- [ ] Contact form has labels and required fields
- [ ] Image has appropriate alternative text
- [ ] Code is properly indented and readable

Submission

Submit your index.html file and the images folder. Be prepared to explain the purpose of the semantic elements used in your project.

MODULE 3: CSS FUNDAMENTALS
CHAPTER 14
INTRODUCTION TO CSS

Learning Objectives

By the end of this chapter, students should be able to:

- Define CSS and explain its role in web development.
- Describe the different ways to apply CSS to HTML.
- Write basic CSS rules using selectors, properties, and values.
- Link an external stylesheet to an HTML document.
- Apply simple styles to text, backgrounds, and page elements.

14.1 What Is CSS?

CSS stands for Cascading Style Sheets. It is a stylesheet language used to control the appearance and layout of HTML webpages.

HTML defines the structure and meaning of content, while CSS controls how that content looks.

CSS can be used to change:

- Text color and font
- Background colors and images
- Element sizes and spacing
- Borders and alignment
- Page layouts
- Responsive behavior on different screen sizes

14.2 Basic CSS Syntax

A CSS rule consists of a selector and a declaration block.

```
selector {
  property: value;
}
```

Example:

```
h1 {
  color: blue;
  font-size: 32px;
}
```

In this example:

- h1 is the selector.
- color and font-size are properties.
- blue and 32px are values.
- Each property-value pair is a declaration.

14.3 Three Ways to Apply CSS

1. Inline CSS

Inline CSS is written directly inside an HTML element using the style attribute.

```
<p style="color: red;">This text is red.</p>
```

Inline CSS is useful for quick demonstrations but is generally not recommended for styling an entire website because it can make code difficult to maintain.

2. Internal CSS

Internal CSS is written inside a <style> element in the HTML document's <head>.

```
<head>
  <style>
    p {
      color: green;
      font-size: 18px;
    }
  </style>
</head>
```

Internal CSS is useful for styling a single webpage.

3. External CSS

External CSS is written in a separate .css file and linked to the HTML document.

HTML file:

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

CSS file (styles.css):

```
p {
  color: purple;
  font-size: 18px;
}
```

External CSS is recommended for most websites because one stylesheet can style multiple pages and makes maintenance easier.

14.4 CSS Selectors

Selectors identify the HTML elements to which CSS rules apply.

Selector	Example	What it selects
Element	p	All paragraph elements
Class	.highlight	Elements with class="highlight"
ID	#header	The element with id="header"
Universal	*	All elements
Group	h1, h2	All <h1> and <h2> elements

Example HTML:

```
<h1 id="header">Welcome</h1>
<p class="highlight">This paragraph is highlighted.</p>
<p>This is a normal paragraph.</p>
```

Example CSS:

```
#header {
  color: navy;
}

.highlight {
  background-color: yellow;
}

p {
  font-family: Arial, sans-serif;
}
```

14.5 Common CSS Properties

```
body {
  background-color: #f4f4f4;
  font-family: Arial, sans-serif;
}

h1 {
  color: darkblue;
  text-align: center;
}

p {
  font-size: 18px;
  line-height: 1.6;
}
```

Common properties include:

- color – sets text color.
- background-color – sets an element's background color.
- font-family – specifies the typeface.
- font-size – controls text size.
- text-align – aligns text.
- line-height – controls spacing between lines of text.
- margin – sets space outside an element.
- padding – sets space inside an element.

14.6 Hands-On Activity

Create two files in the same folder:

```
my-website/
├── index.html
└── styles.css
```

In index.html, create a page containing a heading, two paragraphs, and a link. In styles.css:

- Set a background color for the page.
- Change the heading's color and alignment.
- Set a font family and size for paragraphs.
- Add spacing around the content.
- Style the link with a different color.

Key Terms

- CSS: A stylesheet language used to style HTML.
- Selector: A pattern that identifies HTML elements.
- Property: The style characteristic being changed.
- Value: The setting assigned to a CSS property.
- Stylesheet: A collection of CSS rules.

Review Questions

- What does CSS stand for?
- What is the role of CSS in web development?
- Identify the selector, property, and value in h1 { color: blue; }.
- State the three ways to apply CSS.
- Why is external CSS recommended for multi-page websites?
- What is the difference between a class selector and an ID selector?
- Name four CSS properties and explain their uses.

Practical Assignment

Apply an external stylesheet to your personal portfolio website from the previous module. Style the page background, headings, paragraphs, navigation links, and footer.

CHAPTER 15

CSS COLORS, BACKGROUNDS, AND TYPOGRAPHY

Learning Objectives

By the end of this chapter, students should be able to:

- Identify different ways of specifying colors in CSS.
- Apply background colors and images to HTML elements.
- Control background positioning, repetition, and sizing.
- Understand CSS typography and font properties.
- Style text to improve readability and visual appearance.
- Combine colors, backgrounds, and typography to create attractive webpages.

15.1 Introduction

Colors, backgrounds, and typography are important parts of website design. They determine how a website looks, how readable its content is, and how visitors experience its interface.

CSS provides several properties for controlling these features.

For example, a website can use:

Blue headings to establish visual identity.

Light backgrounds to improve readability.

Larger fonts for important headings.

Different font weights to emphasize information.

Background images to create visual interest.

15.2 CSS Colors

CSS allows developers to specify colors in different formats.

1. Color Names

CSS supports predefined color names.

```
h1 {
  color: blue;
```

```
}

p {
  color: green;
}

body {
  background-color: white;
}
```

Examples of CSS color names include:

- Red
- Blue
- Green
- Yellow
- Black
- White
- Purple
- Orange
- Gray

2. HEX Color Codes

HEX stands for hexadecimal. It represents colors using a six-digit code beginning with #.

Examples:

```
h1 {
  color: #0000FF;
}

p {
  color: #008000;
}

body {
  background-color: #F4F4F4;
}
```

A HEX color generally follows this structure:

#RRGGBB

- RR represents red.

- GG represents green.
- BB represents blue.

Each pair uses hexadecimal values ranging from 00 to FF.

3. RGB Colors

RGB stands for Red, Green, and Blue.

It represents colors using three numerical values, each ranging from 0 to 255.

```
h1 {
  color: rgb(255, 0, 0);
}

p {
  color: rgb(0, 0, 255);
}
```

In the examples:

- rgb(255, 0, 0) produces red.
- rgb(0, 0, 255) produces blue.
- rgb(0, 0, 0) produces black.
- rgb(255, 255, 255) produces white.

4. RGBA Colors

RGBA adds an alpha value to RGB to control transparency.

The alpha value ranges from 0 to 1.

```
.box {
  background-color: rgba(0, 0, 255, 0.5);
}
```

Here, 0.5 represents 50% opacity.

5. HSL Colors

HSL stands for Hue, Saturation, and Lightness.

- Hue: The color itself, represented by an angle from 0 to 360 degrees.

- Saturation: The intensity of the color, represented as a percentage.
- Lightness: How light or dark the color is, represented as a percentage.

Example:

```
h1 {
  color: hsl(240, 100%, 50%);
}
```

This produces blue.

15.3 CSS Background Properties

Background properties control the appearance of an element's background.

1. Background Color

The background-color property sets the background color of an element.

```
body {
  background-color: #f0f0f0;
}

header {
  background-color: navy;
}
```

2. Background Image

The background-image property allows developers to place an image behind an element's content.

```
body {
  background-image: url("images/background.jpg");
}
```

The image should be stored in the correct folder, and the path must point to its location.

3. Background Repeat

The background-repeat property determines whether a background image repeats.

Common values include:

- repeat – repeats horizontally and vertically.
- repeat-x – repeats horizontally.
- repeat-y – repeats vertically.
- no-repeat – displays the image once.

Example:

```
body {
  background-image: url("images/pattern.png");
  background-repeat: no-repeat;
}
```

4. Background Position

The background-position property controls where a background image appears.

```
.hero {
  background-image: url("images/banner.jpg");
  background-position: center;
}
```

Other common values include:

- top
- bottom
- left
- right
- center

5. Background Size

The background-size property controls the size of a background image.

```
.hero {
  background-image: url("images/banner.jpg");
  background-size: cover;
  background-position: center;
```

```
}
```

Common values:

- cover – scales the image to cover the entire element, possibly cropping part of it.
- contain – scales the image so the entire image fits inside the element.
- auto – uses the image's natural dimensions where possible.

6. Background Attachment

The background-attachment property determines how a background image behaves when the page scrolls.

```
body {
  background-image: url("images/background.jpg");
  background-attachment: fixed;
}
```

The fixed value keeps the background image fixed relative to the viewport on browsers that support this behavior.

15.4 CSS Typography

Typography refers to the arrangement and appearance of text on a webpage.

Good typography makes website content easier to read and understand.

1. Font Family

The font-family property specifies the typeface used for text.

```
body {
  font-family: Arial, sans-serif;
}

h1 {
  font-family: Georgia, serif;
}
```

Common font families include:

- Arial

- Georgia
- Verdana
- Times New Roman
- Courier New

A fallback font should be included in case the preferred font is unavailable.

2. Font Size

The font-size property controls the size of text.

```
h1 {
  font-size: 36px;
}

p {
  font-size: 18px;
}
```

CSS supports different units, including:

- px – pixels.
- em – relative to the element's inherited or parent font size.
- rem – relative to the root element's font size.
- % – percentage-based sizing.

3. Font Weight

The font-weight property controls how thick or bold text appears.

```
h1 {
  font-weight: bold;
}

p {
  font-weight: normal;
}
```

Common values include:

- normal
- bold
- 100 to 900, where supported by the selected font.

4. Font Style

The font-style property controls whether text appears normal, italic, or oblique.

```
p {
  font-style: italic;
}
```

5. Text Alignment

The text-align property controls horizontal text alignment.

```
h1 {
  text-align: center;
}

p {
  text-align: left;
}
```

Common values:

- left
- right
- center
- justify

6. Text Decoration

The text-decoration property adds or removes decorative lines.

```
a {
  text-decoration: none;
}

h2 {
  text-decoration: underline;
}
```

Common values include:

- none
- underline
- overline
- line-through

7. Text Transform

The text-transform property changes the capitalization of text.

```
h1 {
  text-transform: uppercase;
}

p {
  text-transform: capitalize;
}
```

Common values:

- uppercase
- lowercase
- capitalize
- none

8. Line Height

The line-height property controls the vertical spacing between lines of text.

```
p {
  line-height: 1.6;
}
```

A comfortable line height improves readability, especially for long paragraphs.

9. Letter Spacing

The letter-spacing property controls the space between individual characters.

```
h1 {
  letter-spacing: 2px;
}
```

15.5 Combining Backgrounds and Typography

A developer can combine several CSS properties to create an attractive webpage.

Example HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>CSS Styling Practice</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <header class="hero">
    <h1>Welcome to My Website</h1>
    <p>Learn, Create, and Innovate.</p>
  </header>

  <main>
    <section>
      <h2>About Us</h2>
      <p>
        We help beginners develop their skills in
        web development and technology.
      </p>
    </section>
  </main>

</body>
</html>

Example CSS (styles.css):

body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  margin: 0;
}

.hero {
  background-color: #123456;
  color: white;
  text-align: center;
  padding: 60px 20px;
}

.hero h1 {
  font-size: 36px;
```

```
  font-weight: bold;
}

.hero p {
  font-size: 18px;
  line-height: 1.6;
}

main {
  padding: 30px;
}

section {
  background-color: white;
  padding: 20px;
}
```

15.6 Introduction to Google Fonts

Google Fonts provides a collection of fonts that developers can use on websites.

For example, to use the Roboto font, include this link inside the HTML <head>:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com"
crossorigin>
<link
href="https://fonts.googleapis.com/css2?family=Roboto:wght@400;700
&display=swap" rel="stylesheet">
```

Then use it in CSS:

```
body {
  font-family: "Roboto", sans-serif;
}
```

Important: External fonts require an internet connection unless the font files are downloaded and hosted locally.

Hands-On Activity

Create a webpage called styling.html and an external stylesheet called styling.css.

Your webpage should contain:

- A header with a heading and tagline.
- An About section.
- A paragraph describing a technology topic.
- A background color for the page.
- A different background color for the header.
- At least two different text colors.
- Different font sizes for headings and paragraphs.
- At least one text alignment property.
- A background image in one section.

Key Terms:

- HEX: A hexadecimal system used to represent colors in CSS.
- RGB: Red, Green, and Blue color representation.
- RGBA: RGB color representation with an alpha transparency value.
- HSL: Hue, Saturation, and Lightness color representation.
- Typography: The arrangement and appearance of text.
- Opacity: The degree to which an element or color is transparent.
- Font family: The typeface used to display text.
- Background image: An image displayed behind an element's content.

Chapter Summary

CSS colors, backgrounds, and typography help developers control the visual appearance of webpages. CSS supports several color formats, including named colors, HEX, RGB, RGBA, and HSL. Background properties control images and colors behind elements, while typography properties control how text appears.

Combining these features helps developers create attractive, readable, and consistent websites.

Review Questions

- What is the purpose of CSS colors?
- Explain the difference between RGB and RGBA.
- What does HEX mean in CSS?
- State five CSS background properties.

- What is the purpose of background-size: cover?
- Define typography.
- Explain the functions of font-family, font-size, and font-weight.
- What is the difference between text-align and text-transform?
- What is the purpose of line height?
- State five best practices for choosing website colors and fonts.

Practical Assignment

Design a technology-themed homepage for KJV Tech using the following requirements:

- Create a dark-colored header.
- Use a suitable background color for the main page.
- Apply a consistent font family.
- Give headings and paragraphs different font sizes.
- Use at least three CSS color formats.
- Include a section with a background image.
- Ensure that text remains readable against its background.

CHAPTER 16

CSS BOX MODEL – MARGIN, PADDING, BORDER, AND CONTENT

Learning Objectives

By the end of this chapter, students should be able to:

- Explain the CSS Box Model.
- Identify the four components of the Box Model.
- Differentiate between margin and padding.
- Apply borders to HTML elements.
- Understand element width and height calculations.
- Use box-sizing to control element dimensions.
- Create properly spaced webpage layouts.

16.1 Introduction to the CSS Box Model

In CSS, every HTML element is treated as a rectangular box.

The CSS Box Model describes how the size of an element is calculated and how space is arranged around it.

Every box consists of four main components:

- Content
- Padding
- Border
- Margin

Understanding the Box Model is important because it helps developers control the size, spacing, and positioning of webpage elements.

16.2 The Four Components of the CSS Box Model

1. Content

Content is the actual information inside an HTML element. It can include text, images, videos, or other elements.

Example:

```
<div class="box">
  Welcome to KJV Tech!
</div>
.box {
```

```
  width: 300px;
  height: 100px;
}
```

Here:

width controls the content width by default.

height controls the content height by default.

2. Padding

Padding is the space between an element's content and its border.

It creates space inside an element.

Example:

```
.box {
  padding: 20px;
}
```

This adds 20 pixels of space around the content, inside the border.

You can also control each side separately:

```
.box {
  padding-top: 10px;
  padding-right: 20px;
  padding-bottom: 15px;
  padding-left: 25px;
}
```

3. Border

A border is the line surrounding an element's padding and content.

Example:

```
.box {
  border: 2px solid black;
}
```

A border declaration generally contains:

- Border width

- Border style
- Border color

Common border styles include:

- solid
- dashed
- dotted
- double
- none

You can style individual sides:

```
.box {
  border-top: 2px solid red;
  border-right: 2px dashed blue;
  border-bottom: 2px solid green;
  border-left: 2px dotted black;
}
```

4. Margin

Margin is the space outside an element's border. It separates one element from other elements.

Example:

```
.box {
  margin: 20px;
}
```

This creates 20 pixels of space outside the element on all four sides.

You can also set individual margins:

```
.box {
  margin-top: 10px;
  margin-right: 20px;
  margin-bottom: 15px;
  margin-left: 25px;
}
```

16.3 Understanding the Difference Between Margin and Padding

Margin and padding both create space, but they work differently.

Margin	Padding
Creates space outside the border	Creates space inside the border
Separates elements from one another	Separates content from its border
Background does not extend into the margin	Background extends behind the padding
Can have collapsing behavior vertically in certain block layouts	Does not collapse

Example:

```
.card {
  background-color: lightblue;
  margin: 20px;
  padding: 30px;
  border: 2px solid blue;
}
```

In this example:

The margin separates the card from surrounding elements.

The padding creates space between the card's content and its border.

The border surrounds the card.

16.4 CSS Shorthand Properties

CSS allows developers to set multiple sides using one declaration.

Padding Shorthand

```
.box {
  padding: 10px 20px 30px 40px;
}
```

The values represent:

- Top: 10px
- Right: 20px
- Bottom: 30px
- Left: 40px

This follows the clockwise order: top, right, bottom, left.

Other shorthand examples:

```
/* All four sides */
.box {
  padding: 20px;
}

/* Top and bottom: 10px; left and right: 20px */
.box {
  padding: 10px 20px;
}
```

Margin Shorthand

The same rules apply to margin:

```
.box {
  margin: 10px 20px 30px 40px;
}
```

To center a block element horizontally when it has an appropriate set width:

```
.container {
  width: 80%;
  margin: 0 auto;
}
```

The auto values distribute the available horizontal space equally on both sides.

16.5 Understanding Element Dimensions

By default, CSS uses the content-box sizing model.

This means that the declared width and height apply only to the content. Padding and borders are added outside those dimensions.

Consider:

```
.box {
  width: 300px;
  padding: 20px;
  border: 5px solid black;
}
```

The actual total width is:

$$ 300 + 20 + 20 + 5 + 5 = 350px $$

Therefore:

Content width = 300px

Left and right padding = 40px

Left and right borders = 10px

Total width = 350px

16.6 The Box-Sizing Property

The box-sizing property determines how an element's width and height are calculated.

There are two common values.

1. Content-Box

This is the default value.

```
.box {
  box-sizing: content-box;
  width: 300px;
  padding: 20px;
  border: 5px solid black;
}
```

Total width = 350px.

2. Border-Box

With border-box, the declared width includes the content, padding, and border.

```
.box {
  box-sizing: border-box;
  width: 300px;
  padding: 20px;
  border: 5px solid black;
}
```

Total width = 300px.

The content area becomes 250px because the padding and border are included within the declared width.

Recommended practice: Many developers use the following rule to make element sizing easier to manage:

```
* {
  box-sizing: border-box;
}
```

This applies border-box to all elements. Some projects also apply it to pseudo-elements using *::before and *::after.

16.7 Practical Example: Creating a Card

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>CSS Box Model</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="card">
    <h2>Welcome to KJV Tech</h2>
    <p>Learn web development and build amazing projects.</p>
    <button>Learn More</button>
  </div>

</body>
</html>
```

CSS (style.css):

```
* {
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
  background-color: #f0f0f0;
}

.card {
  width: 350px;
  background-color: white;
  padding: 25px;
  margin: 50px auto;
  border: 2px solid #123456;
  border-radius: 10px;
}

.card h2 {
  color: #123456;
}

.card p {
  line-height: 1.6;
}

.card button {
  padding: 10px 20px;
  border: none;
  background-color: #123456;
  color: white;
  cursor: pointer;
}
```

This example demonstrates how margin, padding, borders, and box sizing work together to create a card layout.

16.8 Common Mistakes to Avoid

- Confusing margin with padding.
- Forgetting that borders increase the total size under content-box.
- Adding excessive padding that makes content difficult to fit.
- Using inconsistent spacing throughout a website.

- Forgetting to account for borders when calculating element dimensions.
- Using fixed widths without considering smaller screens.
- Applying unnecessary borders to every element.

Hands-On Activity

Create a webpage containing three boxes.

For each box:

- Give it a different background color.
- Apply a border.
- Add padding.
- Add margin.
- Set a width and height.
- Use different border styles.

Then apply box-sizing: border-box and observe how the dimensions change.

Key Terms

- Box Model: The CSS system used to calculate element dimensions and spacing.
- Content: The information inside an HTML element.
- Padding: Space between content and border.
- Border: The line surrounding an element's padding and content.
- Margin: Space outside an element's border.
- Box-sizing: A CSS property that controls how element dimensions are calculated.
- Border-box: A sizing model that includes padding and borders in the declared dimensions.

Chapter Summary

The CSS Box Model explains how every HTML element occupies space on a webpage. It consists of content, padding, border, and margin. Understanding these four components helps developers control element dimensions and spacing. The box-sizing property makes it easier to manage dimensions, especially when building complex layouts.

Review Questions

1. What is the CSS Box Model?

2. List and explain its four components.
3. What is the difference between margin and padding?
4. What is the purpose of the border property?
5. Explain the clockwise order used in CSS shorthand.
6. What is the difference between content-box and border-box?
7. Calculate the total width of an element with a content width of 200px, padding of 15px on each horizontal side, and a 5px border on each horizontal side using content-box.
8. Write a CSS rule that horizontally centers a block element with a width of 80%.

Practical Assignment

Design a simple technology service card for KJV Tech.

Your card must contain:

- A heading.
- A short description.
- A button.
- A background color.
- A visible border.
- Appropriate padding and margin.
- Rounded corners.
- A declared width using box-sizing: border-box.

CHAPTER 17
CSS DISPLAY, POSITIONING, AND OVERFLOW

Learning Objectives

By the end of this chapter, students should be able to:

- Explain the CSS display property.
- Differentiate between block, inline, and inline-block elements.
- Use display: none to hide elements.
- Understand different CSS positioning methods.
- Apply positioning properties to webpage elements.
- Explain how CSS overflow works.
- Create simple layouts using display and positioning properties.

17.1 Introduction

CSS provides different properties for controlling how HTML elements appear, occupy space, and interact with other elements on a webpage.

Three important CSS concepts are:

1. Display: Determines how an element participates in the page layout.
2. Positioning: Controls how an element is placed within a webpage.
3. Overflow: Controls what happens when content exceeds an element's available space.

Understanding these properties helps developers create organized and functional website layouts.

17.2 The CSS Display Property

The display property determines how an element is displayed in a webpage layout.

Some common display values include:

- block
- inline
- inline-block
- none
- flex
- grid

1. Display: Block

A block-level element normally starts on a new line and occupies the available width of its containing block.

Examples of HTML block-level elements include:

-
- <p>
- <h1> to <h6>
- <section>
- <article>

Example:

```
<div class="box">First Box</div>
<div class="box">Second Box</div>
```

CSS:

```
.box {
  display: block;
  background-color: lightblue;
  padding: 20px;
  margin-bottom: 10px;
}
```

The two boxes appear on separate lines.

2. Display: Inline

Inline elements normally remain within the same line as surrounding text.

Examples include:

-
- <a>
-
-

Example:

```
<p>
  Welcome to <span class="highlight">KJV Tech</span>.
  Learn web development.
```

```
</p>
```

CSS:

```
.highlight {
  display: inline;
  color: red;
  font-weight: bold;
}
```

Inline elements generally do not accept width and height in the same way block elements do.

3. Display: Inline-Block

The inline-block value combines characteristics of inline and block elements.

Elements can appear beside one another while still accepting width and height.

Example:

```
<div class="card">HTML</div>
<div class="card">CSS</div>
<div class="card">JavaScript</div>
```

CSS:

```
.card {
  display: inline-block;
  width: 120px;
  padding: 20px;
  margin: 10px;
  background-color: lightblue;
  text-align: center;
}
```

The cards can appear side by side while maintaining their specified dimensions.

4. Display: None

The display: none property removes an element from the page layout.

Example:

```
.hidden {
  display: none;
}
```

HTML:

```
<p class="hidden">This paragraph will not be displayed.</p>
```

Unlike visibility: hidden, which hides an element but preserves its layout space, display: none removes the element and its space from the layout.

17.3 Introduction to CSS Positioning

The CSS position property determines how an element is positioned within a webpage.

The five common positioning values are:

- Static
- Relative
- Absolute
- Fixed
- Sticky

1. Position: Static

Static is the default positioning behavior of HTML elements.

Elements appear in their normal document flow.

```
.box {
  position: static;
}
```

The top, right, bottom, and left properties do not reposition a statically positioned element.

2. Position: Relative

Relative positioning allows an element to be moved from its original position without removing its original space from the document flow.

Example:

```
<div class="box">Relative Box</div>
```

CSS:

```
.box {
  position: relative;
  top: 20px;
  left: 30px;
  background-color: lightblue;
  padding: 20px;
}
```

The element moves 20 pixels down and 30 pixels to the right from its original position.

Its original space remains reserved.

3. Position: Absolute

Absolute positioning removes an element from the normal document flow.

It is positioned relative to its nearest positioned ancestor. If no suitable ancestor exists, it is positioned relative to the initial containing block.

Example:

```
<div class="container">
  <div class="box">Absolute Box</div>
</div>
```

CSS:

```
.container {
  position: relative;
  width: 300px;
  height: 200px;
  background-color: lightgray;
}

.box {
  position: absolute;
  top: 20px;
  right: 20px;
  background-color: blue;
```

```
  color: white;
  padding: 15px;
}
```

Here, the box is positioned 20 pixels from the top and right edges of the container.

4. Position: Fixed

Fixed positioning places an element relative to the viewport. It normally stays in the same viewport position when the page scrolls.

Example:

```
.top-button {
  position: fixed;
  bottom: 20px;
  right: 20px;
  background-color: blue;
  color: white;
  padding: 12px;
}
```

This could be used for a floating navigation or back-to-top control.

5. Position: Sticky

Sticky positioning allows an element to behave like a normally positioned element until a specified scroll position is reached. It then sticks within its scrolling container.

Example:

```
<nav class="navbar">
  <a href="#home">Home</a>
  <a href="#about">About</a>
</nav>
```

CSS:

```
.navbar {
  position: sticky;
  top: 0;
  background-color: navy;
  padding: 15px;
```

```
}
```

The navigation bar sticks to the top when the page is scrolled to its position.

17.4 Understanding Top, Right, Bottom, and Left

The top, right, bottom, and left properties control the offset of positioned elements.

Example:

```
.box {
  position: absolute;
  top: 10px;
  right: 20px;
}
```

This positions the element 10 pixels from the top and 20 pixels from the right edge of its containing block.

These properties do not normally affect statically positioned elements.

17.5 Understanding Z-Index

The z-index property controls the stacking order of overlapping positioned elements and certain other layout items.

A higher stacking value generally places an element in front of an element with a lower value within the same stacking context.

Example:

```
<div class="box box-one">Box One</div>
<div class="box box-two">Box Two</div>
```

CSS:

```
.box {
  position: absolute;
  width: 150px;
  height: 100px;
}

.box-one {
```

```
  background-color: red;
  top: 30px;
  left: 30px;
  z-index: 1;
}

.box-two {
  background-color: blue;
  top: 70px;
  left: 70px;
  z-index: 2;
}
```

Box Two appears above Box One because it has a higher z-index within the same stacking context.

17.6 The CSS Overflow Property

The overflow property controls what happens when content is too large to fit inside its containing element.

Common values include:

- visible
- hidden
- scroll
- auto

1. Overflow: Visible

This is the default behavior. Content can extend beyond the element's box.

```
.container {
  width: 200px;
  height: 100px;
  overflow: visible;
}
```

2. Overflow: Hidden

Content that extends outside the element's box is clipped.

```
.container {
  width: 200px;
  height: 100px;
```

```
  overflow: hidden;
}
```

3. Overflow: Scroll

Scrollbars are provided so users can access content that exceeds the element's available space.

```
.container {
  width: 200px;
  height: 100px;
  overflow: scroll;
}
```

4. Overflow: Auto

The browser provides scrolling when necessary.

```
.container {
  width: 200px;
  height: 100px;
  overflow: auto;
}
```

You can also control horizontal and vertical overflow separately:

```
.container {
  overflow-x: auto;
  overflow-y: hidden;
}
```

17.7 Practical Example: Creating a Navigation Bar

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>CSS Display and Positioning</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
  <header>
    <h1>KJV Tech</h1>
  </header>

  <nav class="navbar">
    <a href="#home">Home</a>
    <a href="#about">About</a>
    <a href="#services">Services</a>
    <a href="#contact">Contact</a>
  </nav>

  <main>
    <section id="home">
      <h2>Welcome to KJV Tech</h2>
      <p>Learn technology, coding, and innovation.</p>
    </section>

    <section id="about">
      <h2>About Us</h2>
      <p>We inspire the next generation of innovators.</p>
    </section>

  </main>

  <footer>
    <p>&copy; 2026 KJV Tech.</p>
  </footer>

</body>
</html>

CSS (style.css):

body {
  font-family: Arial, sans-serif;
  margin: 0;
}

header {
  background-color: navy;
  color: white;
  padding: 20px;
  text-align: center;
}

.navbar {
```

```
  position: sticky;
  top: 0;
  background-color: #222;
  padding: 15px;
}

.navbar a {
  display: inline-block;
  color: white;
  text-decoration: none;
  margin-right: 20px;
}

main {
  padding: 30px;
  min-height: 700px;
}

section {
  padding: 20px;
  margin-bottom: 20px;
  background-color: #f0f0f0;
}

footer {
  background-color: navy;
  color: white;
  text-align: center;
  padding: 15px;
}
```

17.8 Common Mistakes to Avoid

1. Confusing display: none with visibility: hidden.
2. Using absolute positioning for every webpage element.
3. Forgetting to give an absolute element a suitable positioned ancestor when needed.
4. Assuming z-index always works independently of stacking contexts.
5. Using fixed positioning without considering mobile screens.
6. Applying overflow: hidden when users need to access the clipped content.
7. Forgetting that sticky positioning needs an appropriate offset, such as top: 0.
8. Using inline elements when a block or inline-block layout is required.

Hands-On Activity

Create a webpage containing:

- Three boxes displayed side by side.
- One box using display: inline-block.
- A navigation bar using position: sticky.
- A small element positioned absolutely inside a relative container.
- A fixed button at the bottom-right corner.
- A content container demonstrating overflow: auto.

Observe how each property changes the layout.

Key Terms

- Display: Controls how an element participates in the layout.
- Block: An element that normally starts on a new line.
- Inline: An element that normally flows within surrounding text.
- Inline-block: An element that flows inline while accepting block-like dimensions.
- Positioning: The method used to place elements within a layout.
- Z-index: Controls stacking order within stacking contexts.
- Overflow: Controls how content exceeding an element's box is handled.
- Viewport: The visible area of a webpage in a browser.

Chapter Summary

The CSS display property controls how elements participate in a webpage layout. Positioning properties determine where elements appear, while z-index controls their stacking order. The overflow property manages content that exceeds an element's available space.

These concepts help developers build structured, organized, and interactive webpage layouts.

Review Questions

1. What is the purpose of the CSS display property?
2. Explain the difference between block and inline elements.
3. What is the difference between inline and inline-block?
4. What happens when display: none is applied?
5. List the five common CSS positioning values.
6. Explain the difference between relative and absolute positioning.

7. What is the purpose of position: sticky?
8. What does the z-index property control?
9. State four values of the CSS overflow property.
10. What is the difference between overflow: hidden and overflow: auto?

CHAPTER 18
CSS RESPONSIVE WEB DESIGN – MEDIA QUERIES AND MOBILE-FIRST DESIGN

Learning Objectives

By the end of this chapter, students should be able to:

- Explain the meaning and importance of responsive web design.
- Understand how websites adapt to different screen sizes.
- Explain the purpose of CSS media queries.
- Write media queries for different devices.
- Understand the mobile-first design approach.
- Use flexible layouts, images, and typography.
- Apply responsive techniques using Flexbox and CSS Grid.
- Test and improve website responsiveness.

20.1 Introduction to Responsive Web Design

Responsive Web Design (RWD) is an approach to website development that allows web pages to adjust automatically to different screen sizes and devices.

A responsive website should display properly on:

- Desktop computers.
- Laptop computers.
- Tablets.
- Smartphones.

For example, a website may display three columns on a desktop computer but change to one column on a smartphone.

This makes the website easier to use without requiring visitors to zoom in or scroll horizontally.

Responsive web design is important because people access websites using different devices and screen sizes.

20.2 Why Responsive Web Design Is Important

Responsive design is an important part of modern web development for several reasons.

1. Better User Experience

Visitors can read content, view images, and navigate a website comfortably on different devices.

2. Mobile Accessibility

A responsive website makes information accessible to smartphone and tablet users.

3. Improved Website Maintenance

Developers can maintain one responsive website instead of creating separate websites for desktop and mobile users.

4. Better Search Visibility

Search engines consider mobile usability and page experience among their ranking considerations. A mobile-friendly website can support better search performance.

5. Professional Appearance

A responsive website looks organized and professional across different screen sizes.

20.3 Understanding Viewports

A viewport is the visible area of a web page displayed on a device's screen.

Different devices have different viewport widths.

For example:

- A desktop monitor may have a wide viewport.
- A tablet usually has a medium-sized viewport.
- A smartphone usually has a narrow viewport.

To ensure that a website uses the device's actual screen width, developers include the viewport meta tag inside the HTML document's <head> section.

Example:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Understanding the attributes:

width=device-width tells the browser to match the page width to the device's screen width.

initial-scale=1.0 sets the initial zoom level to 100%.

A typical HTML document should include this tag:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Responsive Website</title>
</head>
<body>

    <h1>Welcome to My Website</h1>

</body>
</html>
```

20.4 Introduction to CSS Media Queries

A CSS media query is a CSS technique that allows developers to apply different styles depending on the characteristics of a device or viewport.

For example, a developer can use media queries to:

- Change font sizes on smaller screens.
- Adjust website layouts.
- Hide or display certain elements.
- Change the number of columns.
- Adjust spacing and padding.

The general syntax is:

```
@media (condition) {
    /* CSS rules */
}
```

Example:

```
body {
    background-color: white;
}

@media (max-width: 600px) {
    body {
        background-color: lightblue;
    }
}
```

In this example:

The background is white by default.

When the viewport width is 600 pixels or less, the background changes to light blue.

18.5 Understanding Breakpoints

A breakpoint is a screen-width point at which a website's layout or appearance changes.

Developers use breakpoints to make websites fit different screen sizes.

Common example breakpoints include:

Device Category	Example Breakpoint
Small smartphones	480px
Larger smartphones	600px
Tablets	768px
Small laptops	1024px
Larger screens	1200px

These values are examples, not strict rules. A good developer chooses breakpoints based on when the actual website layout needs to change.

Example:

```
/* Desktop styles */
.container {
```

```
    display: grid;
    grid-template-columns: repeat(3, 1fr);
}

/* Tablet styles */
@media (max-width: 768px) {
    .container {
        grid-template-columns: repeat(2, 1fr);
    }
}

/* Mobile styles */
@media (max-width: 480px) {
    .container {
        grid-template-columns: 1fr;
    }
}
```

This creates:

- Three columns on wider screens.
- Two columns on tablet-sized screens.
- One column on smaller screens.

20.6 Understanding Mobile-First Design

Mobile-first design is a development approach in which developers design and build the mobile version of a website first before expanding the layout for larger screens.

Instead of starting with a complicated desktop layout and reducing it for smartphones, developers begin with a simple layout that works on small screens.

They then add additional styles for tablets and desktops.

Example of mobile-first CSS:

```
.container {
    display: grid;
    grid-template-columns: 1fr;
    gap: 15px;
}

@media (min-width: 600px) {
    .container {
```

```
        grid-template-columns: repeat(2, 1fr);
    }
}

@media (min-width: 900px) {
    .container {
        grid-template-columns: repeat(3, 1fr);
    }
}
```

How it works:

- The default layout displays one column.
- At 600 pixels and above, it displays two columns.
- At 900 pixels and above, it displays three columns.

Advantages of Mobile-First Design

- Encourages simple and organized layouts.
- Helps developers prioritize important content.
- Makes mobile usability a priority.
- Supports progressive enhancement for larger screens.
- Can reduce unnecessary styling and complexity.

20.7 Responsive Images

Images are an important part of website design. However, large images may overflow their containers on smaller screens.

To prevent this, developers can use the following CSS:

```
img {
    max-width: 100%;
    height: auto;
}
```

Explanation:

- max-width: 100% prevents the image from exceeding its container's width.
- height: auto maintains the image's original proportions.

Example:

```
<div class="image-container">
    <img src="technology.jpg" alt="Technology equipment">
```

```
</div>
```

CSS:

```
.image-container {
    width: 100%;
}

.image-container img {
    max-width: 100%;
    height: auto;
    display: block;
}
```

This allows the image to resize according to the available space.

Using the picture Element

HTML also provides the <picture> element, which allows developers to provide different image sources for different screen conditions.

Example:

```
<picture>
    <source media="(max-width: 600px)" srcset="mobile-image.jpg">
    <source media="(min-width: 601px)" srcset="desktop-image.jpg">
    <img src="desktop-image.jpg" alt="Technology workspace">
</picture>
```

The browser selects an appropriate source based on the media conditions.

18.8 Responsive Typography

Responsive typography means adjusting text sizes and spacing to make content readable across different screen sizes.

A heading that looks good on a desktop may be too large on a smartphone.

One way to manage this is through media queries.

Example:

```
h1 {
    font-size: 40px;
```

```
}

@media (max-width: 600px) {
    h1 {
        font-size: 28px;
    }
}
```

You can also use the CSS clamp() function to create flexible font sizes.

Example:

```
h1 {
    font-size: clamp(24px, 5vw, 48px);
}
```

The three values represent:

- 24px: Minimum font size.
- 5vw: Preferred flexible font size based on viewport width.
- 48px: Maximum font size.

This allows the heading to resize within the specified limits.

18.9 Flexible Widths and Heights

Responsive websites should avoid relying entirely on fixed dimensions.

For example:

```
.container {
    width: 1000px;
}
```

This may cause horizontal scrolling on smaller screens.

A more flexible approach is:

```
.container {
    width: 90%;
    max-width: 1000px;
    margin: 0 auto;
}
```

Explanation:

- width: 90% allows the container to use 90% of the available width.
- max-width: 1000px prevents it from becoming too wide.
- margin: 0 auto centers the container horizontally.

Other useful responsive CSS units include:

Unit	Meaning
%	Percentage of a reference size
vw	Percentage of viewport width
vh	Percentage of viewport height
rem	Relative to the root font size
em	Relative to the element's font size
px	Fixed CSS pixel unit

18.10 Responsive Navigation Menus

Navigation menus help visitors move between different pages of a website.

A navigation menu that displays horizontally on a desktop may need to change its arrangement on smaller screens.

Example HTML:

```
<nav class="navbar">
    <h2>KJV Tech</h2>

    <div class="nav-links">
        <a href="#">Home</a>
        <a href="#">About</a>
        <a href="#">Courses</a>
        <a href="#">Contact</a>
    </div>
</nav>
```

CSS:

```
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
```

```
    padding: 15px 5%;
    background-color: navy;
    color: white;
}

.nav-links {
    display: flex;
    gap: 20px;
}

.nav-links a {
    color: white;
    text-decoration: none;
}

@media (max-width: 600px) {
    .navbar {
        flex-direction: column;
        gap: 15px;
    }

    .nav-links {
        flex-direction: column;
        text-align: center;
    }
}
```

In this example:

- The navigation items appear horizontally on wider screens.
- They change to a vertical arrangement on smaller screens.

This is a simple responsive navigation menu. More advanced websites may use a menu button to show and hide navigation links.

18.11 Combining CSS Grid and Flexbox for Responsive Design

CSS Grid and Flexbox can work together to create responsive websites.

Grid is useful for organizing the overall page structure, while Flexbox is useful for arranging elements inside individual sections.

Example:

```
<div class="page">
    <header class="header">
        <h1>KJV Tech</h1>
    </header>

    <main class="content">
        <section class="cards">
            <div class="card">HTML</div>
            <div class="card">CSS</div>
            <div class="card">JavaScript</div>
        </section>
    </main>
</div>
```

CSS:

```
* {
    box-sizing: border-box;
}

.page {
    display: grid;
    grid-template-columns: 1fr;
    gap: 20px;
    padding: 20px;
}

.header {
    background-color: navy;
    color: white;
    padding: 20px;
}

.cards {
    display: flex;
```

```
    flex-wrap: wrap;
    gap: 15px;
}

.card {
    flex: 1 1 200px;
    background-color: lightblue;
    padding: 30px;
    text-align: center;
    border-radius: 8px;
}

@media (min-width: 768px) {
    .page {
        grid-template-columns: 1fr 3fr;
    }
}
```

This example demonstrates how both layout systems can be combined to create flexible designs.

18.12 Testing Website Responsiveness

After creating a responsive website, developers must test it to ensure that it works correctly on different devices.

One useful tool is the browser's Developer Tools.

Testing with Google Chrome:

- Open your website in Google Chrome.
- Right-click anywhere on the page.
- Select Inspect.
- Locate the device toolbar icon.
- Click the device toolbar.
- Select different device sizes or enter custom dimensions.
- Observe how the website responds.

You can also use the keyboard shortcut:

Ctrl + Shift + I

Then activate the device toolbar using:

Ctrl + Shift + M

Check the following during testing:

- Does the website fit the screen?
- Are the texts readable?
- Do images resize properly?
- Does the navigation menu work?
- Are buttons easy to use?
- Is there unwanted horizontal scrolling?
- Do the layouts adjust correctly?

Testing should be performed at multiple widths, not just on one smartphone or desktop size.

18.13 Common Responsive Design Mistakes

Some common mistakes beginners make include:

- Forgetting to include the viewport meta tag.
- Using fixed widths that are too large for small screens.
- Allowing images to overflow their containers.
- Using text that is too small on mobile devices.
- Ignoring navigation usability on smartphones.
- Applying media queries without testing the actual layout.
- Designing only for desktop screens.
- Forgetting to test buttons and other interactive elements.

Developers should always test their websites on different viewport sizes and make adjustments where necessary.

18.14 Hands-On Activity

Create a responsive personal profile webpage.

Your webpage should contain:

- A header with your name or brand name
- A navigation menu.
- A profile image.
- An introduction section.
- A section showing your skills.
- A contact section.
- A footer.

Requirements:

- Use HTML5 to structure the page.
- Use CSS to style the webpage.
- Apply mobile-first design.
- Use media queries to adjust the layout.
- Make all images responsive.
- Ensure that text is readable on mobile devices.
- Test the website using Chrome Developer Tools.

18.15 Key Terms

Term	Meaning
Responsive Web Design	An approach to building websites that adapt to different screen sizes.
Viewport	The visible area of a webpage on a device.
Media Query	A CSS technique for applying styles under specified conditions.
Breakpoint	A viewport size at which the layout changes.
Mobile-First Design	An approach that starts with the mobile layout before expanding to larger screens.
Flexible Layout	A layout that adjusts to available space.
clamp()	A CSS function that sets minimum, preferred, and maximum values.
max-width	Sets the maximum width an element can occupy.
Developer Tools	Browser tools used to inspect and test websites.

18.16 Chapter Summary

In this chapter, we learned that responsive web design allows websites to adapt to different screen sizes and devices.

We explored the viewport, CSS media queries, breakpoints, mobile-first design, responsive images, flexible typography, and responsive navigation menus.

We also learned how to combine CSS Grid and Flexbox to create flexible layouts and how to test websites using browser Developer Tools.

Responsive design is an essential skill for every modern web developer because websites must provide a good experience across smartphones, tablets, and desktop computers.

18.18 Practical Assignment

Build a fully responsive website for a fictional technology training organisation called KJV Tech.

Your website should contain:

- A responsive navigation bar.
- A hero section introducing the organisation.
- An About Us section.
- A Courses section with at least three course cards.
- A section showing the benefits of learning technology.
- A contact section.
- A footer.

Technical Requirements:

- Use HTML5 for the page structure.
- Use CSS3 for styling.
- Apply mobile-first design.
- Include at least two media queries.
- Make images and text responsive.
- Test the website on mobile, tablet, and desktop screen sizes.

Expected Outcome: Students should be able to build a website that adjusts its layout and appearance to provide a good user experience across different devices.

CHAPTER 19
JAVASCRIPT FUNDAMENTALS

Introduction

JavaScript is one of the most important programming languages used in modern web development. It enables developers to create interactive, dynamic, and responsive websites. While HTML provides the structure of a webpage and CSS controls its appearance, JavaScript adds functionality and behaviour.

With JavaScript, a website can respond when a user clicks a button, validate information entered into a form, display messages, update content without reloading the page, create animations, and communicate with external services.

This chapter introduces the fundamental concepts of JavaScript and guides students through practical examples that demonstrate how the language is used in web development.

19.1 Understanding JavaScript

JavaScript is a high-level programming language primarily used to make websites interactive and dynamic. It allows developers to write instructions that a computer executes to perform specific tasks.

For example, consider a webpage containing a button labelled Click Me. HTML creates the button, CSS styles it, and JavaScript determines what happens when a visitor clicks it.

Example:

```
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Example</title>
</head>
<body>

    <h1>Welcome to JavaScript</h1>

    <button onclick="alert('Hello, Student!')">
        Click Me
    </button>
```

```
</body>
</html>
```

When the user clicks the button, JavaScript displays a message.

19.1.1 Importance of JavaScript

JavaScript is important because it:

- Makes websites interactive.
- Responds to user actions.
- Validates form information.
- Updates webpage content dynamically.
- Creates animations and visual effects.
- Communicates with APIs and external services.
- Supports the development of web applications.
- Can also be used for server-side programming through environments such as Node.js.

19.2 Features and Uses of JavaScript

JavaScript has several features that make it useful in modern software development.

1. Lightweight: It can execute many programming tasks without requiring a large amount of system resources.
2. Interpreted and just-in-time compiled: Modern JavaScript engines process and optimise code as it runs.
3. Dynamic: Variables can hold different types of values during program execution.
4. Event-driven: Programs can respond to actions such as clicks, typing, and form submissions.
5. Object-based: It supports objects and object-oriented programming concepts.
6. Cross-platform: It runs in modern browsers and several other programming environments.
7. Asynchronous: It supports operations that can complete later, such as retrieving information from a server.

19.2.1 Where JavaScript Is Used

JavaScript is used in:

- Website development
- Interactive dashboards.
- Online registration systems.

- E-commerce applications.
- Web-based games.
- Mobile application development.
- Desktop applications.
- Server-side applications.
- Real-time communication applications.

19.3 Setting Up a JavaScript Development Environment

Before writing JavaScript, students need a suitable development environment.

Required Tools

- A computer.
- A modern web browser such as Google Chrome, Microsoft Edge, or Firefox.
- A code editor such as Visual Studio Code.
- Basic knowledge of HTML and CSS.

Creating a JavaScript Project

Create a folder named javascript-practice.

Inside the folder, create three files:

```
javascript-practice/
│
├── index.html
├── style.css
└── script.js
```

The files serve different purposes:

index.html contains the webpage structure.

style.css contains the webpage styling.

script.js contains the JavaScript code.

19.4 How JavaScript Runs in a Browser

A web browser contains a JavaScript engine responsible for processing and executing JavaScript code.

When a browser loads a webpage, it reads the HTML, processes the CSS, and executes JavaScript when the browser reaches the relevant script instructions.

For example, Google Chrome uses the V8 JavaScript engine.

JavaScript can be included in an HTML document in three main ways.

19.4.1 Inline JavaScript

Inline JavaScript is written directly inside an HTML element using an event attribute.

```
<button onclick="alert('Welcome!')">
    Click Here
</button>
```

This method is simple but is generally not recommended for larger projects because it mixes HTML and JavaScript.

19.4.2 Internal JavaScript

Internal JavaScript is written inside a script element in the HTML document.

```
<!DOCTYPE html>
<html>
<head>
    <title>Internal JavaScript</title>
</head>
<body>

    <h1>Learning JavaScript</h1>

    <script>
        console.log("Hello from JavaScript!");
    </script>

</body>
</html>
```

19.4.3 External JavaScript

External JavaScript involves writing code in a separate .js file and connecting it to the HTML document.

In index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>External JavaScript</title>
    <script src="script.js" defer></script>
</head>
<body>

    <h1>Learning External JavaScript</h1>

</body>
</html>
```

In script.js:

```
console.log("JavaScript is working!");
```

The defer attribute instructs the browser to execute the external script after the HTML document has been parsed.

Best practice: Use external JavaScript files for most projects because they make code easier to organise, maintain, and reuse.

19.5 JavaScript Syntax, Statements, and Comments

Syntax refers to the rules that determine how instructions must be written in a programming language.

JavaScript is case-sensitive. This means that uppercase and lowercase letters are treated differently.

For example:

```
let studentName = "Joshua";

console.log(studentName);
```

The following would produce an error if StudentName has not been declared:

```
console.log(StudentName);
```

This is because studentName and StudentName are different identifiers.

19.5.1 Statements

A statement is an instruction that tells JavaScript to perform an action.

```
let age = 20;
console.log(age);
```

The first statement creates a variable, while the second displays its value.

19.5.2 Expressions

An expression is a piece of code that produces a value.

```
10 + 5
```

Another example:

```
let total = 10 + 5;
```

The expression 10 + 5 produces the value 15.

19.5.3 Semicolons

Semicolons are used to mark the end of JavaScript statements.

```
let name = "Joshua";
let age = 20;

console.log(name);
console.log(age);
```

JavaScript can automatically insert semicolons in many situations, but using them consistently can make code clearer.

19.5.4 Code Blocks

A code block is a group of statements enclosed in curly braces { }.

```
if (10 > 5) {
    console.log("Ten is greater than five.");
}
```

The statements inside the braces belong to the conditional block.

19.5.5 Comments

Comments are notes written inside code for explanation. JavaScript ignores comments during normal execution.

Single-line comment:

```
// This is a single-line comment

let name = "Joshua";
```

Multi-line comment:

```
/*
This is a multi-line comment.
It can contain several lines.
*/

let age = 20;
```

Comments help developers understand their programs, especially when working on large projects.

19.6 Variables in JavaScript

A variable is a named storage location used to hold data that a program can access and work with.

For example:

```
let studentName = "Joshua";
let age = 20;

console.log(studentName);
console.log(age);
```

JavaScript provides three main keywords for declaring variables: var, let, and const.

19.6.1 The let Keyword

The let keyword declares a variable whose value can be changed.

```
let score = 50;
score = 75;
console.log(score);
```

Output:

75

19.6.2 The const Keyword

The const keyword declares a variable that cannot be reassigned after it has been initialised.

```
const school = "Federal University of Technology";

console.log(school);
```

Trying to reassign it will produce an error:

```
const country = "Nigeria";
country = "Ghana"; // Error
```

A const object or array can still have its contents modified; what cannot change is the variable's binding to that object or array.

19.6.3 The var Keyword

The var keyword is an older way of declaring variables.

```
var course = "Web Development";

console.log(course);
```

It is still supported, but modern JavaScript generally favours let and const because their scope rules are more predictable.

Difference Between var, let, and const

Feature	var	let	const
Can be reassigned	Yes	Yes	No
Can be redeclared in the same scope	Yes	No	No
Block-scoped	No	Yes	Yes
Modern recommendation	Avoid generally	Use when reassignment is needed	Prefer by default

19.6.4 Variable Naming Rules

When naming variables:

- A variable name must not begin with a number.
- Spaces are not allowed.
- Reserved JavaScript keywords cannot be used as variable names.
- Variable names are case-sensitive.
- Names should be meaningful.
- Letters, numbers, underscores, and dollar signs may be used, subject to the first-character rule.

Examples:

```
let firstName = "Joshua";
let studentAge = 20;
let total_score = 85;
let $price = 500;
```

A common naming convention is camelCase, where the first word begins with a lowercase letter and subsequent words begin with uppercase letters.

19.7 JavaScript Data Types

A data type describes the kind of value stored in a variable.

JavaScript has several built-in data types.

19.7.1 String

A string is text enclosed in quotation marks.

```
let name = "Joshua";
let school = 'FUTA';
```

19.7.2 Number

The number type represents both whole numbers and decimal numbers.

```
let age = 20;
let price = 2500.50;
```

19.7.3 Boolean

A Boolean represents either true or false.

```
let isRegistered = true;
let isGraduated = false;
```

19.7.4 Undefined

A variable that has been declared but not assigned a value contains undefined.

```
let email;

console.log(email);
```

Output:

```
undefined
```

19.7.5 Null

null represents the intentional absence of a value.

```
let middleName = null;
```

19.7.6 Object

An object stores related information using key-value pairs.

```
let student = {
    name: "Joshua",
    age: 20,
    course: "Computer Science"
};
```

19.7.7 Array

An array stores multiple values in an ordered collection.

```
let subjects = [
    "HTML",
    "CSS",
    "JavaScript"
];
```

19.7.8 Checking Data Types

The typeof operator can be used to identify the type of a value.

```
let name = "Joshua";
let age = 20;
let isStudent = true;

console.log(typeof name);
console.log(typeof age);
console.log(typeof isStudent);

Output:

string
number
boolean

One special detail: typeof null returns "object" due to a
historical JavaScript behaviour.
```

19.8 JavaScript Operators and Expressions

Operators are symbols used to perform operations on values and variables.

19.8.1 Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Remainder	10 % 3	1
**	Exponentiation	2 ** 3	8

```
Example:

let firstNumber = 20;
let secondNumber = 10;

console.log(firstNumber + secondNumber);
console.log(firstNumber - secondNumber);
console.log(firstNumber * secondNumber);
console.log(firstNumber / secondNumber);
```

19.8.2 Assignment Operators

Assignment operators assign values to variables.

```
let score = 10;

score += 5; // score = score + 5
score -= 2; // score = score - 2
score *= 2; // score = score * 2

console.log(score);
```

19.8.3 Comparison Operators

Comparison operators compare two values and return a Boolean result.

Operator	Meaning
==	Equal value
===	Equal value and equal type
!=	Not equal value
!==	Not equal value or type
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
let age = 18;

console.log(age >= 18);
console.log(age === 18);
```

Output:

```
true
true
```

The strict equality operator === is generally preferred because it compares both value and data type.

19.8.4 Logical Operators

Logical operators combine or reverse conditions.

|

Operator	Meaning
&&	AND
||	OR
!	NOT

Example:

```
let age = 20;
let hasID = true;

console.log(age >= 18 && hasID);
```

The result is true because both conditions are true.

19.8.5 Increment and Decrement Operators

The increment operator ++ increases a value by one.

The decrement operator -- decreases a value by one.

```
let number = 5;

number++;
console.log(number);

number--;
console.log(number);
```

19.8.6 String Concatenation

The + operator can join strings.

```
let firstName = "Joshua";
let lastName = "Victor";

let fullName = firstName + " " + lastName;
```

```
console.log(fullName);
```

Output:

```
Joshua Victor
```

19.8.7 Template Literals

Template literals use backticks and allow variables to be inserted using ${ }.

```
let name = "Joshua";
let age = 20;

console.log(`My name is ${name} and I am ${age} years old.`);
```

Template literals are useful for creating readable messages and combining text with values.

19.9 Conditional Statements

Conditional statements allow a program to make decisions based on whether a condition is true or false.

19.9.1 The if Statement

The if statement executes a block of code when a condition is true.

```
let age = 20;

if (age >= 18) {
    console.log("You are an adult.");
}
```

19.9.2 The if...else Statement

The else block executes when the condition in the if statement is false.

```
let score = 65;

if (score >= 50) {
    console.log("You passed.");
} else {
    console.log("You failed.");
}
```

19.9.3 The else if Statement

The else if statement checks additional conditions.

```
let score = 85;

if (score >= 70) {
    console.log("Grade A");
} else if (score >= 60) {
    console.log("Grade B");
} else if (score >= 50) {
    console.log("Grade C");
} else {
    console.log("Fail");
}
```

19.9.4 The switch Statement

The switch statement checks a value against several possible cases.

```
let day = 3;

switch (day) {
    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
        break;

    case 3:
        console.log("Wednesday");
        break;

    default:
        console.log("Invalid day");
}
```

The break statement prevents execution from continuing into the next case.

19.9.5 The Ternary Operator

The ternary operator provides a shorter way to write a simple if...else expression.

```
let age = 20;

let message = age >= 18
    ? "Eligible"
    : "Not eligible";

console.log(message);
```

19.10 Loops in JavaScript

Loops allow a program to execute a block of code repeatedly while a specified condition is satisfied.

They reduce the need to write the same instruction many times.

19.10.1 The for Loop

The for loop is commonly used when the number of repetitions is known.

```
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

Output:

```
1
2
3
4
5
```

The loop contains three main parts:

- Initialisation: let i = 1
- Condition: i <= 5
- Update: i++

19.10.2 The while Loop

A while loop continues running as long as its condition is true.

```
let i = 1;

while (i <= 5) {
    console.log(i);
```

```
    i++;
}
```

19.10.3 The do...while Loop

A do...while loop executes its code at least once before checking the condition.

```
let number = 1;

do {
    console.log(number);
    number++;
} while (number <= 5);
```

19.10.4 The for...of Loop

The for...of loop is used to iterate over values in iterable collections such as arrays.

```
let courses = ["HTML", "CSS", "JavaScript"];

for (let course of courses) {
    console.log(course);
}
```

19.10.5 The break and continue Statements

break terminates a loop.

```
for (let i = 1; i <= 10; i++) {
    if (i === 5) {
        break;
    }

    console.log(i);
}
```

continue skips the current iteration and moves to the next one.

```
for (let i = 1; i <= 5; i++) {
    if (i === 3) {
        continue;
    }

    console.log(i);
```

```
}
```

19.11 Functions in JavaScript

A function is a reusable block of code designed to perform a particular task.

Functions help developers organise code, reduce repetition, and make programs easier to maintain.

19.11.1 Declaring a Function

A function can be declared using the function keyword.

```
function greet() {
    console.log("Welcome to KJV Tech!");
}
```

19.11.2 Calling a Function

Declaring a function does not automatically execute it. It must be called.

```
function greet() {
    console.log("Welcome to KJV Tech!");
}

greet();
```

19.11.3 Function Parameters

Parameters are variables listed in a function declaration. They receive values when the function is called.

```
function greetStudent(name) {
    console.log(`Welcome, ${name}!`);
}

greetStudent("Joshua");
greetStudent("David");
```

19.11.4 Function Arguments

Arguments are the actual values supplied when calling a function.

In the example above, "Joshua" and "David" are arguments.

19.11.5 The return Statement

The return statement sends a result back from a function.

```
function addNumbers(a, b) {
    return a + b;
}

let result = addNumbers(10, 5);

console.log(result);
```

Output:

```
15
```

19.11.6 Function Expressions

A function can also be stored inside a variable.

```
const greet = function() {
    console.log("Hello Student!");
};

greet();
```

19.11.7 Arrow Functions

Arrow functions provide a shorter syntax for writing functions.

```
const add = (a, b) => {
    return a + b;
};

console.log(add(5, 3));
```

For a single expression, the return can be implicit:

```
const multiply = (a, b) => a * b;

console.log(multiply(4, 5));
```

19.11.8 Scope of Variables

Scope determines where a variable can be accessed.

- Global scope: A variable declared outside functions and blocks can generally be accessed throughout its applicable script or module
- Function scope: A variable declared with var inside a function is accessible within that function.
- Block scope: Variables declared with let and const inside a block are accessible within that block.

Example:

```
let school = "FUTA";

function showSchool() {
    let department = "Computer Science";

    console.log(school);
    console.log(department);
}

showSchool();
```

The school variable is accessible inside the function because it was declared outside it. The department variable is limited to the function.

19.12 Arrays in JavaScript

An array is a data structure used to store multiple values in a single variable.

```
let students = [
    "Joshua",
    "David",
    "Mary",
    "John"
];
```

19.12.1 Accessing Array Elements

Array positions are called indexes. JavaScript arrays use zero-based indexing, meaning the first element is at index 0.

```
console.log(students[0]);
console.log(students[1]);
```

Output:

```
Joshua
David
```

19.12.2 Changing Array Elements

```
students[1] = "Daniel";

console.log(students);
```

19.12.3 The Length Property

The length property returns the number of elements in an array.

```
console.log(students.length);
```

19.12.4 Common Array Methods

Method	Purpose
push()	Adds an element to the end
pop()	Removes the last element
shift()	Removes the first element
unshift()	Adds an element to the beginning
includes()	Checks whether a value exists
indexOf()	Finds the index of a value
slice()	Returns part of an array
splice()	Adds or removes elements
map()	Creates a new array from transformed values
filter()	Creates a new array containing matching values
forEach()	Executes a function for each element

Example:

```
let courses = ["HTML", "CSS"];

courses.push("JavaScript");

console.log(courses);
```

Output:

```
["HTML", "CSS", "JavaScript"]
```

19.12.5 Using map() and filter()

The map() method transforms every element in an array.

```
let numbers = [1, 2, 3, 4];

let doubled = numbers.map(number => number * 2);

console.log(doubled);
```

The filter() method selects elements that satisfy a condition.

```
let scores = [45, 70, 85, 30, 60];

let passed = scores.filter(score => score >= 50);

console.log(passed);
```

19.13 Objects in JavaScript

An object is a collection of related data and functionality organised as key-value pairs.

Objects are useful when representing real-world entities such as students, products, employees, or customers.

Example:

```
let student = {
    name: "Joshua",
    age: 20,
    department: "Computer Science",
    school: "FUTA"
};
```

19.13.1 Accessing Object Properties

Properties can be accessed using dot notation or bracket notation.

```
console.log(student.name);
console.log(student["department"]);
```

19.13.2 Modifying Object Properties

```
student.age = 21;

console.log(student.age);
```

19.13.3 Adding Object Properties

```
student.level = 200;

console.log(student);
```

19.13.4 Object Methods

An object method is a function stored as a property of an object.

```
let student = {
    name: "Joshua",

    introduce: function() {
        console.log(`My name is ${this.name}.`);
    }
};

student.introduce();
```

The this keyword refers to the object in the context of this method call.

19.14 Introduction to the Document Object Model (DOM)

The Document Object Model, commonly called the DOM, is a programming interface that represents an HTML document as a structured tree of objects.

JavaScript uses the DOM to access and manipulate webpage elements.

For example, JavaScript can:

- Change text.
- Change colours and styles.
- Add or remove elements.
- Change image sources.
- Respond to user actions.
- Modify attributes.

Consider this HTML:

```
<h1 id="heading">Welcome</h1>
```

JavaScript can access the heading using:

```
let heading = document.getElementById("heading");

console.log(heading);
```

19.14.1 Common DOM Selection Methods

Method	Purpose
getElementById()	Selects an element by ID
getElementsByClassName()	Selects elements by class
getElementsByTagName()	Selects elements by tag name
querySelector()	Selects the first matching CSS selector
querySelectorAll()	Selects all matching CSS selectors

Example:

```
const heading = document.querySelector("h1");
const button = document.querySelector(".my-button");
const paragraph = document.getElementById("description");
```

19.14.2 Changing HTML Content

The textContent property changes the text inside an element.

```
<h1 id="heading">Original Heading</h1>

<script>
    document.getElementById("heading").textContent =
        "New Heading";
</script>
```

The innerHTML property can insert or read HTML markup. It should be used carefully with trusted content because inserting untrusted HTML can create security vulnerabilities.

19.14.3 Changing CSS Styles

JavaScript can change the style of an element.

```
<p id="message">Learning JavaScript</p>
```

```
<script>
    const message = document.getElementById("message");

    message.style.color = "red";
    message.style.fontSize = "25px";
</script>
```

19.14.4 Adding and Removing HTML Elements

JavaScript can create new elements and insert them into a webpage.

```
<div id="container"></div>

<script>
    const container = document.getElementById("container");

    const paragraph = document.createElement("p");

    paragraph.textContent = "This paragraph was created using
JavaScript.";

    container.appendChild(paragraph);
</script>
```

To remove an element:

```
paragraph.remove();
```

19.15 JavaScript Events and Event Listeners

An event is an action or occurrence that a browser can detect.

Examples include:

- Clicking a button.
- Moving a mouse.
- Typing on a keyboard
- Submitting a form.
- Loading a webpage
- Changing an input field.

JavaScript uses event listeners to respond to these actions.

19.15.1 Click Event

```
<button id="myButton">Click Me</button>

<script>
    const button = document.getElementById("myButton");

    button.addEventListener("click", function() {
        alert("Button clicked!");
    });
</script>
```

19.15.2 Input Event

The input event occurs whenever the value of an input field changes.

```
<input type="text" id="nameInput" placeholder="Enter your name">

<p id="output"></p>

<script>
    const input = document.getElementById("nameInput");
    const output = document.getElementById("output");

    input.addEventListener("input", function() {
        output.textContent = input.value;
    });
</script>
```

19.15.3 Submit Event

The submit event occurs when a form is submitted.

```
<form id="registrationForm">
    <input type="text" id="studentName" placeholder="Your name">
    <button type="submit">Register</button>
</form>

<script>
    const form = document.getElementById("registrationForm");

    form.addEventListener("submit", function(event) {
        event.preventDefault();

        alert("Form submitted successfully!");
```

```
    });
</script>
```

The preventDefault() method prevents the browser's default form-submission behaviour. This is useful when JavaScript needs to process the form first.

1.16 Form Validation and User Input

Form validation is the process of checking information entered by users to ensure that it meets specified requirements.

For example, a registration form may require:

- Full name.
- Email address.
- Phone number
- Password.
- Confirmation of required information.

JavaScript can check these values before allowing a form to proceed.

19.16.1 Reading Input Values

The value property retrieves the information entered into an input field.

```
<input type="text" id="name" placeholder="Enter your name">

<button id="submitButton">Submit</button>

<script>
    const nameInput = document.getElementById("name");
    const submitButton = document.getElementById("submitButton");

    submitButton.addEventListener("click", function() {
        console.log(nameInput.value);
    });
</script>
```

19.16.2 Basic Form Validation

```
<form id="myForm">
    <input type="text" id="name" placeholder="Enter your name">
```

```
    <input type="email" id="email" placeholder="Enter your email">

    <button type="submit">Submit</button>
</form>

<p id="message"></p>

<script>
    const form = document.getElementById("myForm");
    const nameInput = document.getElementById("name");
    const emailInput = document.getElementById("email");
    const message = document.getElementById("message");

    form.addEventListener("submit", function(event) {
        event.preventDefault();

        if (
            nameInput.value.trim() === "" ||
            emailInput.value.trim() === ""
        ) {
            message.textContent = "Please fill in all fields.";
            return;
        }

        message.textContent = "Registration information received!";
    });
</script>
```

This example checks that the name and email fields are not empty.

Important: Browser-side validation improves user experience, but it does not replace server-side validation. Information submitted to a real application must also be checked by the server.

19.17 Error Handling and Debugging

An error is a problem in a program that prevents it from behaving as expected.

Debugging is the process of identifying, understanding, and correcting errors.

19.17.1 Common JavaScript Errors

Syntax Error: Occurs when code violates the language's grammatical rules.

```
let name = "Joshua"
console.log(name;
```

The missing closing parenthesis causes an error.

Reference Error: Occurs when code attempts to access a variable that has not been declared in the accessible scope.

```
console.log(studentName);
```

If studentName has not been declared, JavaScript reports a reference error.

Type Error: Occurs when an operation is performed on a value in an invalid way.

Logical Error: Occurs when the code runs but produces an incorrect result because the instructions or logic are wrong.

19.17.2 Using the Console

The browser developer console is an important debugging tool.

JavaScript provides several console methods.

```
console.log("Normal message");

console.info("Information message");

console.warn("Warning message");

console.error("Error message");
```

To open the developer tools in most desktop browsers:

- Open the webpage.
- Press F12 or Ctrl + Shift + I.
- Select the Console tab.
- Examine the messages and errors displayed.

19.17.3 Using try...catch

The try...catch statement helps a program handle certain runtime errors without immediately stopping the entire script.

```
try {
```

```
    let result = unknownFunction();

    console.log(result);
} catch (error) {
    console.log("An error occurred.");
}
```

The catch block runs when an error is thrown inside the try block.

19.18 JSON and Working with Data

JSON stands for JavaScript Object Notation. It is a lightweight text format used for storing and exchanging data between applications.

JSON is commonly used when websites communicate with servers and APIs.

Example of JSON data:

```
{
    "name": "Joshua",
    "age": 20,
    "course": "Computer Science"
}
```

19.18.1 Converting JavaScript Objects into JSON

The JSON.stringify() method converts a JavaScript value into a JSON string.

```
const student = {
    name: "Joshua",
    age: 20
};

const studentJSON = JSON.stringify(student);

console.log(studentJSON);
```

1.18.2 Converting JSON into JavaScript Objects

The JSON.parse() method converts valid JSON text into a JavaScript value.

```
const studentJSON = '{"name":"Joshua","age":20}';
```

```
const student = JSON.parse(studentJSON);

console.log(student.name);
```

JSON is useful when transferring information between front-end and back-end applications.

19.19 Asynchronous JavaScript

Asynchronous JavaScript allows certain operations to take place without requiring the program to wait for them to finish before continuing with other work.

This is important when working with

- Server requests.
- APIs.
- File operations.
- Timers.
- Database-related services.

19.19.1 The setTimeout() Function

setTimeout() schedules a function to execute after a specified delay.

```
console.log("First message");

setTimeout(function() {
    console.log("Second message");
}, 2000);

console.log("Third message");
```

The third message appears before the second because the second is scheduled to execute after approximately two seconds.

19.19.2 Callbacks

A callback is a function passed into another function to be executed later or when a particular operation is completed.

```
function greetStudent(name, callback) {
    console.log(`Hello, ${name}`);
```

```
    callback();
}

function finishGreeting() {
    console.log("Welcome to the class!");
}

greetStudent("Joshua", finishGreeting);
```

19.19.3 Promises

A Promise represents the eventual completion or failure of an asynchronous operation.

A Promise can be in one of three states.

- Pending.
- Fulfilled.
- Rejected.

Example:

```
const myPromise = new Promise((resolve, reject) => {
    let success = true;

    if (success) {
        resolve("Operation completed!");
    } else {
        reject("Operation failed!");
    }
});

myPromise
    .then(result => console.log(result))
    .catch(error => console.log(error));
```

19.19.4 Async and Await

The async and await keywords provide a readable way to work with Promises.

```
function getMessage() {
    return new Promise(resolve => {
        setTimeout(() => {
            resolve("Welcome to KJV Tech!");
        }, 2000);
```

```
    });
}

async function displayMessage() {
    const message = await getMessage();

    console.log(message);
}

displayMessage();
```

An async function returns a Promise, while await pauses execution within that asynchronous function until the awaited Promise settles.

19.20 Fetch API and External Data

The Fetch API allows JavaScript to make network requests and retrieve information from external resources such as web APIs.

For example, a website can retrieve information about users, products, weather, or news from a server.

19.20.1 Basic Fetch Request

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => {
        if (!response.ok) {
            throw new Error("Network response was not successful");
        }

        return response.json();
    })
    .then(data => {
        console.log(data);
    })
    .catch(error => {
        console.error("Error:", error);
    });
```

19.20.2 Using Fetch with Async and Await

```
async function getUsers() {
    try {
        const response = await fetch(
            "https://jsonplaceholder.typicode.com/users"
```

```
        );

        if (!response.ok) {
            throw new Error("Unable to retrieve users");
        }

        const users = await response.json();

        console.log(users);

    } catch (error) {
        console.error(error);
    }
}

getUsers();
```

The example retrieves sample user information from a public testing API.

A real application should handle network errors, unsuccessful HTTP responses, and unexpected data carefully.

19.21 Best Practices in JavaScript Programming

Good programming habits help developers write clean, reliable, and maintainable code.

- Use meaningful variable and function names.
- Prefer const unless a variable needs reassignment.
- Use let when reassignment is necessary.
- Avoid unnecessary use of global variables.
- Maintain consistent indentation and formatting.
- Write comments where explanations are useful.
- Separate JavaScript from HTML and CSS where practical.
- Use strict equality (===) when appropriate.
- Validate user input.
- Handle errors properly.
- Test code regularly.
- Avoid inserting untrusted content using innerHTML.
- Organise large programs into reusable functions and modules.
- Do not expose passwords, secret API keys, or private credentials in browser-side JavaScript.

1.22 Practical Projects

The following projects will help students apply the concepts learned in this chapter.

Project 1: Interactive Welcome Page

Objective: Create a webpage that welcomes a visitor by name.

Requirements:

- Create a heading.
- Add a name input field.
- Add a button.
- When the visitor enters a name and clicks the button, display a personalised welcome message.
- Display an error message if the input is empty.

Project 2: Simple Calculator

Objective: Build a calculator that performs basic mathematical operations.

Requirements:

Two number input fields.

Addition button.

Subtraction button.

Multiplication button.

Division button.

Display the result.

Handle division by zero.

19.24 Key JavaScript Terms

Term	Meaning
JavaScript	A programming language used to create interactive applications
Variable	A named storage location for data

Syntax	Rules governing how code is written
Data Type	The classification of a value
Operator	A symbol used to perform an operation
Condition	A statement that determines whether code should execute
Loop	A structure that repeats instructions
Function	A reusable block of code
Array	An ordered collection of values
Object	A collection of related properties and values
DOM	The programming interface representing an HTML document
Event	An action or occurrence detected by the browser
Event Listener	Code that waits for and responds to an event
Debugging	The process of identifying and correcting errors
JSON	A text format used to exchange data
API	An interface that allows software applications to communicate
Promise	An object representing the eventual result of an asynchronous operation
Fetch API	A browser interface for making network requests

Conclusion

JavaScript transforms static webpages into interactive applications. Every serious front-end developer should understand its fundamentals before progressing to advanced frameworks and libraries.

Students are encouraged to practise consistently, experiment with code, solve programming problems, and build small projects. The ability to apply what has been learned is more important than simply memorising programming concepts

CHAPTER 20
GIT AND GITHUB

Introduction

Software development involves writing, modifying, testing, and improving code over time. As a project grows, developers may need to correct errors, introduce new features, experiment with different ideas, or work with other developers.

Keeping track of these changes manually can become difficult. This is where Git and GitHub become important.

Git is a version control system that helps developers record changes made to files and manage different versions of a project. GitHub is an online platform that hosts Git repositories and provides tools for collaboration, project management, code review, and software development.

Although Git and GitHub are closely related, they are not the same thing. Git is the tool used to track changes, while GitHub is a platform that allows developers to store and collaborate on Git projects online.

This chapter explains the fundamentals of Git and GitHub, from installation and basic commands to branching, collaboration, and publishing projects.

20.1 Understanding Version Control

Version control is a system used to record and manage changes made to files over time.

It allows developers to maintain different versions of a project, review previous changes, and return to an earlier version when necessary.

Imagine a developer creating a website. The developer initially creates a homepage, later adds a contact page, and eventually redesigns the navigation menu.

Without version control, managing these changes may become difficult. The developer might save several copies of the same project with names such as:

website-final/

website-final-new/
website-final-updated/
website-final-corrected/
website-final-latest/

This approach is disorganised and can lead to confusion.

Version control provides a more reliable way to manage project history.

20.1.1 Importance of Version Control

Version control is important because it:

- Tracks changes made to project files.
- Records who made particular changes.
- Allows developers to review previous versions.
- Helps recover from unwanted changes.
- Supports collaboration between developers.
- Makes it easier to experiment with new features.
- Helps organise software development projects.
- Provides a history of project development.

20.1.2 Types of Version Control Systems

Version control systems are commonly grouped into three categories:

1. Local Version Control System

This stores versions of files on a single computer.

2. Centralised Version Control System

This uses a central server where developers access and manage shared project versions.

3. Distributed Version Control System

This allows each developer to have a complete local copy of a repository and its history.

Git is a distributed version control system.

20.2 Understanding Git

Git

Distributed version control system

Git tracks changes in files and helps developers manage different versions of their projects.

Git is a free and open-source distributed version control system used to track changes in source code and other files.

It was originally created by Linus Torvalds in 2005 to support the development of the Linux kernel.

Git works locally on a developer's computer. This means developers can record changes, create branches, and review project history without necessarily being connected to the internet.

20.2.1 Features of Git

Some important features of Git include:

- Distributed: Each developer can have a complete copy of the repository and its history.
- Fast: Git can perform many version control operations efficiently
- Branching: Developers can work on separate versions of a project.
- Merging: Changes from different branches can be combined.
- History tracking: Git records commits and project changes.
- Offline operation: Many Git operations work without an internet connection.
- Open-source: Git is freely available and can be used for personal and commercial projects.

2.3 Understanding GitHub

GitHub

Online code hosting and collaboration platform

GitHub provides online repositories and tools for managing and collaborating on software projects.

GitHub is an online platform built around Git. It allows developers to host repositories remotely, share their work, review code, manage projects, and collaborate with other people.

Developers can create public repositories that others can view or private repositories with restricted access.

GitHub is widely used by individual developers, organisations, technology companies, and open-source communities.

2.3.1 Features of GitHub

Important GitHub features include:

- Online repository hosting.
- Collaboration between developers.
- Pull requests and code reviews.
- Issue tracking.
- Project management.
- Branch management.
- GitHub Actions for workflow automation.
- GitHub Pages for publishing supported static websites.
- Documentation using README files.
- Access control for private projects.

2.4 Difference Between Git and GitHub

Although Git and GitHub work together, they serve different purposes.

Git	GitHub
A version control system	An online code hosting platform
Runs locally on a computer	Accessed through the internet and supported applications
Tracks changes in files	Hosts and shares Git repositories
Works offline for many operations	Online access is needed for most platform features
Created by Linus Torvalds	Launched by GitHub's founding team in 2008
Uses commands such as git commit	Provides features such as pull requests and issues

Example: A developer uses Git to record changes made to a website. The developer then pushes the repository to GitHub so that other developers can access and collaborate on the project.

20.5 Installing Git

Before using Git, students must install it on their computers.

Git is available for Windows, macOS, and Linux.

20.5.1 Installing Git on Windows

Follow these steps:

- Open a web browser.
- Visit the official Git website: git-scm.com.
- Select the download option for Windows.
- Download the installer.
- Open the downloaded installer.
- Follow the installation instructions.
- Use the recommended settings unless your instructor specifies otherwise.
- Complete the installation.

20.5.2 Verifying Git Installation

After installation

- Open Command Prompt, PowerShell, or Git Bash.
- Enter the following command:

```
git --version
```

If Git is installed correctly, the terminal should display an installed version, for example:

```
git version 2.x.x
```

The exact version will depend on the installed release.

20.5.3 Understanding Git Bash

Git Bash is a command-line environment installed with Git for Windows.

It allows developers to execute Git commands and many familiar Unix-style terminal commands.

Developers can also use Git through PowerShell, Command Prompt, Visual Studio Code's integrated terminal, and other compatible terminals.

20.6 Configuring Git for the First Time

After installing Git, developers should configure their identity.

This information is associated with commits they create.

Open your terminal and enter:

```
git config --global user.name "Your Full Name"
```

Next, configure your email:

```
git config --global user.email "your-email@example.com"
```

Replace the sample details with your own information.

20.6.1 Checking Git Configuration

To view your configured username:

```
git config --global user.name
```

To view your configured email:

```
git config --global user.email
```

To display the available configuration settings:

```
git config --list
```

20.6.2 Understanding the --global Option

The --global option applies the configuration to all Git repositories associated with your user account on that computer.

Without --global, configuration can be applied only to the current repository.

For example:

```
git config user.name "Project Developer"
```

This changes the username for the current repository.

20.7 Understanding Git Repositories

A repository, commonly called a repo, is a storage location where Git manages a project's files and version history.

A repository contains the project files and Git information used to track changes.

There are two common repository locations:

1. Local Repository

A repository stored on a developer's computer.

2. Remote Repository

A repository hosted on another system or online platform, such as GitHub.

20.7.1 Creating a Project Folder

Create a folder for your project.

For example:

```
my-first-website/
```

Inside this folder, create an HTML file:

```
my-first-website/
└── index.html
```

20.7.2 Initialising a Git Repository

Open the terminal inside the project folder and execute:

```
git init
```

This command initialises a new Git repository in the current directory.

Git creates a hidden directory named .git, which stores repository information and history.

20.7.3 Checking Repository Status

Use:

git status

This command shows the current state of your working directory and staging area.

It helps you identify files that have been modified, added, or are not yet tracked by Git.

2.8 Understanding the Git Workflow

Git uses a workflow that allows developers to prepare, record, and manage changes.

The three major areas are:

- Working Directory.
- Staging Area.
- Repository.

The Git workflow

Working Directory

Where files are created and modified

git add

Staging Area

Changes selected for the next commit

git commit

Local Repository

Recorded project history and commits

git push

Remote Repository

Repository hosted on GitHub or another Git server

20.8.1 Working Directory

The working directory is the project folder where developers create and edit files.

For example, modifying index.html changes the working directory.

20.8.2 Staging Area

The staging area contains changes selected for inclusion in the next commit.

The git add command moves selected changes into the staging area.

20.8.3 Repository

The repository stores recorded changes as commits.

The git commit command creates a record of the staged changes.

20.9 Essential Git Commands

Git commands are instructions entered into a terminal to perform version control operations.

The following commands are among the most frequently used.

Command	Purpose
git init	Creates a new repository
git status	Displays the repository status
git add	Stages changes
git commit	Records staged changes
git log	Displays commit history
git diff	Displays differences between changes
git branch	Lists or manages branches
git switch	Switches between branches
git merge	Combines branch changes
git clone	Copies a remote repository

git push	Sends local commits to a remote repository
git pull	Fetches and integrates remote changes
git fetch	Downloads remote updates without integrating them
git restore	Restores files to an earlier state
git rm	Removes tracked files
git remote	Manages remote repository connections

20.9.1 The git add Command

The git add command stages files for the next commit.

To stage one file:

```
git add index.html
```

To stage all changes in the current directory:

```
git add .
```

20.9.2 The git commit Command

A commit records staged changes in the repository.

```
git commit -m "Create homepage"
```

The -m option allows a commit message to be supplied directly.

A good commit message should describe the changes made.

Examples:

```
git commit -m "Add navigation menu"
git commit -m "Fix contact form validation"
git commit -m "Update homepage styling"
```

20.9.3 The git log Command

The git log command displays the history of commits.

```
git log
```

For a shorter view:

```
git log --oneline
```

2.9.4 The git diff Command

The git diff command displays differences between versions of files.

```
git diff
```

To inspect staged changes:

```
git diff --staged
```

2.9.5 The git restore Command

The git restore command can discard unstaged changes in a file.

```
git restore index.html
```

Important: This can permanently discard uncommitted work in that file. Students should understand the command before using it.

2.10 Working with GitHub

After learning the basic Git commands, students can begin using GitHub to host and manage their projects online.

2.10.1 Creating a GitHub Account

Open Source MOOC | Task 1
opensciencemooc.github.io

To create an account:

1. Open a web browser.
2. Visit GitHub.
3. Select Sign Up.

6.
7.

Enter your email address.

8.
9.

Create a password.

10.
11.

Choose a username.

12.
13.

Complete the verification process.

14.
15.

Follow the instructions to finish creating the account.

16.

After registration, sign in to access your GitHub dashboard.

2.10.2 Understanding the GitHub Dashboard

The GitHub dashboard provides access to repositories, notifications, projects, and other platform features.

GitHub for Everyone – git and GitHub
ucsb-library-research-data-services.github.io

Students should become familiar with:

•

Profile.

•
•

Repositories.

-
-

Issues.

-
-

Pull requests.

-
-

Projects.

-
-

Settings.

-
-

Notifications.

-

2.10.3 Creating a GitHub Repository

A GitHub repository is an online location where a project and its version history can be stored.

To create one:

1.

Sign in to GitHub.

2.
3.

Select the new repository option.

4.
5.

Enter a repository name.

6.
7.

Add an optional description.

8.
9.

Choose Public or Private.

10.
11.

Optionally initialise it with a README file.

12.
13.

Select Create Repository.

14.

[Introduction to Versioning & Git – IS310 – Culture As Data Spring 2026
cultureasdata-uiuc.github.io]

Public repository: Can be viewed by anyone.

Private repository: Access is restricted to the owner and people granted permission.

2.11 Connecting a Local Repository to GitHub

A developer can connect a local Git repository to a remote GitHub repository.

For this example, assume the local project folder is named my-first-website.

Step 1: Initialise the Local Repository

```
git init
```

Step 2: Add Project Files

```
git add .
```

Step 3: Create the First Commit

```
git commit -m "Initial project setup"
```

Step 4: Rename the Main Branch

```
git branch -M main
```

The main branch is commonly used as the primary branch of a repository.

Step 5: Connect the Remote Repository

Copy the HTTPS repository address from GitHub.

Example:

```
git remote add origin https://github.com/username/my-first-website.git
```

Replace username and the repository name with the correct details.

Step 6: Push the Project to GitHub

```
git push -u origin main
```

This sends the local commits to the remote repository and establishes an upstream relationship for the branch.

After the first push, subsequent pushes can usually be made with:

```
git push
```

2.12 Understanding Remote Repositories

A remote is a reference to another repository, usually hosted on a server or online platform.

The most common remote name is origin.

2.12.1 Viewing Remote Repositories

```
git remote -v
```

This displays the remote repository addresses used for fetching and pushing.

2.12.2 Adding a Remote Repository

```
git remote add origin https://github.com/username/project.git
```

2.12.3 Changing a Remote Repository Address

```
git remote set-url origin https://github.com/username/new-project.git
```

2.12.4 Removing a Remote

```
git remote remove origin
```

This removes the remote reference from the local repository. It does not delete the repository hosted on GitHub.

2.13 Cloning a GitHub Repository

Cloning means creating a local copy of an existing remote repository.

It is useful when developers want to download a project, contribute to an existing project, or continue working on a project from another computer.

2.13.1 Cloning Using HTTPS

Open the GitHub repository and copy its HTTPS address.

Then enter:

```
git clone https://github.com/username/project.git
```

Git downloads the repository and its history into a new local directory.

2.13.2 Opening the Cloned Project

Move into the downloaded folder:

```
cd project
```

Open the project in Visual Studio Code:

```
code .
```

The code . command works when the Visual Studio Code command-line launcher is available in the terminal.

20.14 Pushing and Pulling Changes

Developers frequently need to send changes to GitHub and retrieve changes made by other people.

Two important commands are git push and git pull.

20.14.1 Git Push

The git push command sends local commits to a remote repository.

Example:

```
git add .
git commit -m "Update website homepage"
git push
```

20.14.2 Git Pull

The git pull command fetches changes from a remote repository and integrates them into the current branch.

Example:

```
git pull origin main
```

This is useful when another developer has made changes to the remote repository.

20.14.3 Difference Between Push and Pull

Git Push	Git Pull
Sends local commits to a remote repository	Retrieves and integrates remote changes
Updates the remote repository	Updates the local branch

Commonly used after committing	Commonly used before continuing collaborative work

20.14.4 Understanding Git Fetch

The git fetch command downloads updates from a remote repository without automatically integrating them into the current branch.

```
git fetch origin
```

It allows developers to inspect incoming changes before deciding how to integrate them.

20.15 Branching in Git

A branch is an independent line of development within a Git repository.

Branches allow developers to work on different features without directly changing the main branch.

For example, a developer building a website may create:

main – the primary project branch.

feature-login – a branch for developing the login feature.

feature-contact – a branch for developing the contact page.

fix-navigation – a branch for correcting navigation issues.

20.15.1 Creating a Branch

```
git branch feature-login
```

20.15.2 Listing Branches

```
git branch
```

The current branch is marked with an asterisk.

20.15.3 Switching Branches

```
git switch feature-login
```

20.15.4 Creating and Switching to a Branch

```
git switch -c feature-contact
```

This creates a new branch and switches to it.

20.15.5 Deleting a Branch

```
git branch -d feature-login
```

This deletes a branch that Git considers safely merged.

20.15.6 Pushing a Branch to GitHub

```
git push -u origin feature-login
```

This publishes the branch to the remote repository.

20.16 Merging Branches

Merging is the process of combining changes from one branch into another.

For example, after completing a login feature, a developer may want to combine it with the main project.

The process is:

Step 1: Switch to the Main Branch

```
git switch main
```

Step 2: Merge the Feature Branch

```
git merge feature-login
```

If Git can combine the changes automatically, the merge will complete.

Step 3: Push the Updated Main Branch

```
git push origin main
```

The feature branch can be deleted after it is no longer needed.

20.17 Understanding Merge Conflicts

A merge conflict occurs when Git cannot automatically combine changes from different branches.

This can happen when two developers modify the same section of a file in different ways.

For example, two developers may change the same heading in index.html.

Git may display a conflict similar to:

```
<<<<<<< HEAD
<h1>Welcome to KJV Tech</h1>
=======
<h1>Welcome to Our Website</h1>
>>>>>>> feature-homepage
```

These markers identify the conflicting sections.

2.17.1 Resolving a Merge Conflict

To resolve the conflict:

- Open the affected file.
- Examine the conflicting changes.
- Decide which version to keep or combine the changes.
- Remove the conflict markers.
- Save the file.
- Stage the resolved file.
- Complete the merge commit if Git requires one.

Example of a resolved heading:

```
<h1>Welcome to KJV Tech Web Development</h1>
```

Then run:

```
git add index.html
git commit
```

If the merge is still in progress, Git will complete it using the commit command.

Important: Always understand the changes before resolving a conflict. Do not automatically delete another developer's work.

20.18 Pull Requests on GitHub

A pull request is a GitHub feature that allows developers to propose changes from one branch to another.

It is commonly used in collaborative software development.

A login feature can create a pull request to propose merging that feature into the main branch.

20.18.1 Creating a Pull Request

- Push the feature branch to GitHub.
- Open the repository on GitHub.
- Select Pull Requests.
- Select New Pull Request.
- Choose the base branch, usually main.
- Choose the branch containing the proposed changes.
- Review the differences.
- Add a descriptive title and explanation.
- Create the pull request.

20.18.2 Reviewing a Pull Request

Other developers can:

- Examine the changes.
- Add comments.
- Suggest improvements.
- Request further changes.
- Approve the proposed changes.

After the review process, an authorised user can merge the pull request.

20.19 Forking a Repository

Forking means creating your own GitHub copy of another user's repository under your account.

Forking is especially useful when contributing to open-source projects.

20.19.1 How to Fork a Repository

- Open the GitHub repository.
- Select the Fork option.
- Choose your GitHub account as the destination.
- Create the fork.

You now have a separate repository under your account.

20.19.2 Cloning Your Fork

```
git clone https://github.com/your-username/project.git
```

You can make changes locally and push them to your fork.

20.19.3 Forking and Cloning Compared

Forking	Cloning
Creates a separate repository under your GitHub account	Creates a local copy on your computer
Takes place on GitHub	Takes place using Git
Useful for contributing to projects	Useful for working on repositories locally

20.20 Working with README Files

A README file provides important information about a project.

It is commonly written in Markdown and stored in the root directory of a repository.

The standard filename is:

README.md

20.20.1 Importance of README Files

A README file helps visitors understand.

- What the project does.
- Why the project was created.
- How to install or run it.
- What technologies were used.
- How to contribute.
- Who developed the project.

20.20.2 Example README File

```
# KJV Tech Website

## About
A technology education website developed by KJV Tech.

## Technologies Used
- HTML
- CSS
- JavaScript

## Features
- Homepage
- About section
- Technology articles
- Learning resources
- Contact information

## Installation
1. Clone the repository.
2. Open the project folder.
3. Open index.html in a browser.

## Developer
KJV Tech
```

20.20.3 Basic Markdown Syntax

Markdown is a lightweight formatting language commonly used in README files.

Markdown	Result
# Heading	Main heading
## Subheading	Secondary heading
Bold text	Bold text
Italic text	Italic text
- Item	Bullet list
[Text](URL)	Hyperlink
`code`	Inline code

20.21 Using .gitignore

A .gitignore file tells Git which untracked files and directories to ignore.

Not every project file should be committed to a repository.

For example, developers may want to exclude:

- Temporary files.
- Generated files.
- Dependency folders.
- Environment configuration files.
- Local editor settings.

20.21.1 Creating a .gitignore File

Create a file named:

.gitignore

Example:

```
node_modules/
.env
dist/
*.log
.DS_Store
```

This tells Git to ignore matching untracked files and directories.

2.21.2 Why .gitignore Is Important

It helps to:

- Keep repositories organised.
- Avoid uploading unnecessary files.
- Reduce repository size.
- Prevent accidental exposure of environment configuration files.
- Make projects easier for other developers to download and use.

Important: .gitignore does not automatically remove files that have already been committed. If a sensitive file was previously committed or pushed, additional steps are needed to remove it from tracking and address the exposure. Changing a password or API key may also be necessary.

20.22 Collaborating with Other Developers

Git and GitHub make it possible for several developers to work on the same project.

For example, a team building a website may divide responsibilities as follows:

Team Member	Responsibility
Developer 1	Homepage
Developer 2	Contact page
Developer 3	Registration form
Developer 4	Website styling
Team Lead	Reviewing and merging changes

20.22.1 Basic Collaboration Workflow

A typical workflow involves:

- Cloning the shared repository.
- Creating a new branch.
- Making changes to project files.
- Testing the changes.
- Committing the changes.
- Pushing the branch to GitHub.
- Creating a pull request.
- Reviewing the proposed changes.
- Merging the approved changes.

20.22.2 Good Collaboration Practices

- Communicate with team members
- Avoid editing the same sections without coordination.
- Use meaningful branch names.
- Write descriptive commit messages.
- Pull recent changes before starting new work.
- Review code before merging.
- Document important project decisions.
- Never share account passwords or private access tokens.

20.23 GitHub Authentication and Security

GitHub requires authentication when users perform operations that need access to repositories.

For HTTPS-based Git operations, GitHub supports authentication through approved methods such as Git Credential Manager and personal access tokens.

For SSH-based Git operations, developers can configure an SSH key pair.

20.23.1 HTTPS Authentication

When connecting through HTTPS, developers may use Git Credential Manager or an appropriately configured personal access token.

GitHub no longer accepts ordinary account passwords for Git operations over HTTPS.

20.23.2 SSH Authentication

SSH allows a developer's computer to authenticate with GitHub using a configured SSH key.

The public key is added to the GitHub account, while the private key remains securely on the developer's computer.

20.23.3 Security Best Practices

- Use a strong, unique account password.
- Enable two-factor authentication.
- Never publish private credentials.
- Avoid committing .env files containing secrets.
- Use access tokens with appropriate permissions.
- Review repository access permissions.
- Remove credentials that are no longer needed.
- Protect private SSH keys.

20.24 Publishing Websites with GitHub Pages

GitHub Pages is a hosting service that allows users to publish supported static websites directly from GitHub repositories.

It is suitable for projects built with technologies such as:

- HTML.
- CSS.
- JavaScript.

It does not directly execute server-side PHP or Node.js applications.

20.24.1 Preparing a Website

Create a simple project:

```
my-website/
│
├── index.html
├── style.css
└── script.js
```

Ensure that the homepage is named index.html.

20.24.2 Uploading the Project to GitHub

Initialise the repository, commit the files, connect the GitHub remote, and push the project.

20.24.3 Enabling GitHub Pages

- Open the repository on GitHub.
- Select Settings.
- Find the Pages section.
- Choose the deployment source.
- Select the appropriate branch and folder.
- Save the configuration.
- Wait for the deployment process to complete.

GitHub will provide a website address when the deployment succeeds.

For more information, visit Official GitHub Pages documentation.

20.25 GitHub Issues and Project Management

GitHub provides project management features that help developers organise work and track problems.

20.25.1 Issues

An issue is used to report a bug, suggest a feature, ask a question, or track a task.

For example:

- Fix the navigation menu.
- Add a registration form.
- Improve mobile responsiveness
- Correct the footer layout.

2.25.2 GitHub Projects

GitHub Projects helps teams organise tasks and monitor progress.

A team may create columns such as:

- To Do.
- In Progress.
- Under Review.
- Completed.

These tools help development teams coordinate their work.

20.26 Advanced Git Concepts: A Brief Introduction

After learning the essential Git commands, students should become familiar with several additional concepts.

20.26.1 Git Tags

A tag identifies a specific point in a repository's history, often used to mark software releases.

```
git tag v1.0.0
```

To push a tag:

```
git push origin v1.0.0
```

20.26.2 Git Stash

The git stash command temporarily stores changes that have not been committed.

```
git stash
```

To restore the most recently stashed changes:

```
git stash pop
```

This can be useful when a developer needs to switch tasks without immediately committing unfinished work.

20.26.3 Git Revert

The git revert command creates a new commit that reverses the changes introduced by an earlier commit.

```
git revert COMMIT_ID
```

Replace COMMIT_ID with the appropriate commit identifier.

This is often useful for safely undoing changes in a shared project history.

20.26.4 Git Reset

The git reset command moves the current branch reference to a different commit and can also change the staging area or working directory depending on the option used.

Example:

```
git reset --soft HEAD~1
```

This moves the current branch back by one commit while keeping the changes staged.

Students should understand the consequences of reset options before using them, particularly in shared repositories.

20.27 Git and GitHub Best Practices

Professional developers follow good practices to keep repositories organised and secure.

1. Commit changes regularly.
2. Write clear commit messages.
3. Use descriptive branch names.
4. Keep the main branch stable.
5. Test code before pushing changes.
6. Use .gitignore appropriately.
7. Write a useful README file.

8. Review pull requests before merging.
9. Avoid committing unnecessary files.
10. Never publish passwords or secret keys.
11. Keep project documentation updated.
12. Learn how to resolve merge conflicts.
13. Pull relevant remote changes before beginning collaborative work.
14. Use branches for new features and bug fixes.
15. Make meaningful commits rather than committing every tiny action without context.

20.28 Practical Projects

Students should complete the following projects to demonstrate their understanding of Git and GitHub.

Create and Manage Your First Git Repository

Objective: Learn how to create a local Git repository and track project changes.

Requirements:

- Create a folder named my-first-git-project.
- Create an index.html file.
- Initialise Git inside the project folder.
- Check the repository status.
- Stage the project files.
- Create an initial commit.
- Modify the HTML file.
- Stage and commit the new changes.
- View the commit history.

Upload a Website to GitHub

Objective: Learn how to upload a local web development project to GitHub.

Requirements:

- Create a simple website using HTML and CSS.

- Create a GitHub account.
- Create a public repository.
- Initialise Git in the local project.
- Commit the project files.
- Connect the local repository to GitHub.
- Push the project to GitHub.
- Confirm that the files appear in the online repository.

20.30 Common Git and GitHub Errors

Students may encounter errors while using Git. Understanding their causes makes troubleshooting easier.

Error or Message	Possible Cause	Suggested Solution
git is not recognized	Git is not installed or unavailable in PATH	Install Git and reopen the terminal
Author identity unknown	Git username or email is not configured	Configure user.name and user.email
nothing to commit	No new changes are available to commit	Check the working directory
remote origin already exists	A remote named origin is already configured	Check with git remote -v
failed to push some refs	Remote contains changes not integrated locally, or another push issue exists	Inspect the error and integrate the appropriate remote changes
fatal: not a git repository	Terminal is not inside a Git repository	Navigate to the correct project folder
merge conflict	Git cannot automatically combine changes	Review and resolve the conflicting files
Authentication failed	Authentication credentials are missing or invalid	Configure an approved authentication method

Important: Error messages should be read carefully. Different causes can produce similar Git errors, so students should investigate the actual message instead of running random commands.

20.31 Key Git and GitHub Terms

Term	Meaning
Version	A system for tracking changes to files

Control	
Git	A distributed version control system
GitHub	An online platform for hosting and collaborating on Git repositories
Repository	A project location managed by Git
Commit	A recorded set of changes
Working Directory	The local project files being edited
Staging Area	The area containing changes prepared for a commit
Branch	An independent line of development
Merge	Combining changes from different branches
Clone	Creating a local copy of a remote repository
Push	Sending local commits to a remote repository
Pull	Fetching and integrating remote changes
Fetch	Downloading remote updates without integrating them
Fork	Creating a separate GitHub copy of another repository
Pull Request	A proposal to merge changes into another branch
Merge Conflict	A situation where Git cannot automatically combine changes
README	A document explaining a project
.gitignore	A file specifying which untracked files Git should ignore
Remote	A reference to another repository
GitHub Pages	A service for publishing supported static websites
Git Stash	A feature for temporarily storing uncommitted changes
Git Tag	A named reference to a specific point in project history

20.32 Chapter Summary

Git and GitHub are essential tools in modern software development. They help developers track changes, manage project versions, collaborate with other programmers, and share their work.

Understanding Git and GitHub gives developers the ability to manage their projects professionally and participate effectively in real-world software development teams.

REFERENCES AND RECOMMENDED LEARNING RESOURCES

This manual has been prepared to introduce students to the fundamental principles, tools, technologies, and practices involved in modern web development. Students are encouraged to consult additional learning materials to strengthen their understanding, explore advanced topics, and remain informed about changes in the technology industry.

The following resources are recommended for further study.

Books and Online Learning Platforms

1. MDN Web Docs
A comprehensive learning resource for HTML, CSS, JavaScript, web APIs, accessibility, and other web technologies.
Website: https://developer.mozilla.org/

2. W3Schools
Provides beginner-friendly tutorials, examples, and references for HTML, CSS, JavaScript, PHP, SQL, MySQL, and other programming technologies.
Website: https://www.w3schools.com/

3. freeCodeCamp
Offers free programming lessons, practical exercises, coding projects, and certifications in different areas of software development.
Website: https://www.freecodecamp.org/

4. Git Documentation
Provides guides and references for understanding Git commands, repositories, branches, and version control workflows.
Website: https://git-scm.com/doc

5. GitHub Docs
Explains how to use GitHub for hosting repositories, collaborating on projects, managing code, and publishing websites.
Website: https://docs.github.com/

6. The Odin Project
Provides a structured curriculum with practical assignments and projects for aspiring web developers.
Website: https://www.theodinproject.com/

1.2 Official Technical Documentation

Students should develop the habit of reading official documentation because it provides reliable information about how programming languages, frameworks, and development tools work.

Recommended documentation includes:

Technology	Recommended documentation
HTML	https://developer.mozilla.org/en-US/docs/Web/HTML
CSS	https://developer.mozilla.org/en-US/docs/Web/CSS
JavaScript	https://developer.mozilla.org/en-US/docs/Web/JavaScript
Git	https://git-scm.com/doc
GitHub	https://docs.github.com/

These resources complement the lessons in this manual and provide opportunities for continuous learning.

GLOSSARY OF IMPORTANT TECHNICAL TERMS

The following terms are commonly used in web development. Students should become familiar with them as they continue their learning journey.

Term	Meaning
Algorithm	A step-by-step procedure for solving a problem.
API	A set of rules that allows different software applications to communicate.
Backend	The server-side part of an application that handles processing and data.
Browser	Software used to access and display websites.
Client	A device or application that requests services from a server.
CSS	A stylesheet language used to control the appearance of web pages.
Debugging	The process of identifying and correcting errors in a program.
Domain Name	The human-readable address used to access a website.
Frontend	The part of a website or application that users interact with directly.
Git	A version control system used to track changes in files and projects.
GitHub	An online platform for hosting and collaborating on Git repositories.
HTML	The markup language used to structure web page content.
Hosting	A service that makes a website or application accessible online.
JavaScript	A programming language used to add functionality and interactivity to websites.
Responsive Design	An approach to designing websites that adapt to different screen sizes.
Server	A computer or software system that provides services or resources to clients.
URL	The address used to locate a resource on the internet.
UI	User Interface; the visual elements through which

	users interact with an application.
UX	User Experience; the overall experience a person has when using a product.
Version Control	A system for recording and managing changes to files over time.
Web Development	The process of creating, maintaining, and improving websites and web applications.

3. GENERAL CONCLUSION

Web development is an important part of the modern digital world. From business websites and educational platforms to online banking systems, communication applications, and digital marketplaces, web technologies continue to influence how individuals, organisations, and communities interact.

Throughout this course, students have been introduced to the fundamental technologies and practical skills required to begin their journey into web development. They have explored how websites are structured, styled, and made interactive, how information is processed and stored, and how developers manage and organise their projects.

The knowledge gained from studying HTML, CSS, JavaScript, Git, GitHub provides a foundation upon which students can build more advanced technical skills.

However, completing this manual should not be considered the end of the learning process. Technology changes continuously, and successful developers must be willing to learn, practise, experiment, and improve.

Students are therefore encouraged to:

- Continue practising the concepts taught in this manual.
- Build personal and collaborative projects
- Create and maintain professional portfolios.
- Explore additional programming languages and frameworks.
- Read technical documentation and follow industry developments.
- Participate in developer communities and technology events.
- Seek internships, mentorship, and practical experience.
- Use technology to identify problems and develop meaningful solutions.

Remember that becoming a skilled developer is a gradual process. Every project completed, every error corrected, and every new concept understood contributes to professional growth.

The goal is not simply to learn how to write code but to develop the ability to use technology to solve real-world problems.

FINAL MESSAGE TO STUDENTS

YOUR JOURNEY STARTS HERE!

Completing this course is an important milestone, but it is only the beginning of your journey.

Do not be discouraged when your code does not work as expected. Do not allow difficult concepts to convince you that you cannot become a developer. Every experienced programmer was once a beginner.

Be curious. Ask questions. Practise consistently. Build projects. Learn from your mistakes and never stop exploring.

The world needs creative thinkers, problem-solvers, innovators, and individuals who can use technology to make a difference.

You have taken the first step. Keep learning, keep building, and keep creating.

Your ideas can become solutions. Your skills can create opportunities. Your creativity can shape the future.

KJV TECH

Inspiring the Next Generation of Innovators.

ACKNOWLEDGEMENTS

KJV Tech sincerely appreciates everyone who contributed directly or indirectly to the development of this Web Development Course Manual.

Special appreciation goes to the instructors, technology professionals, educational contributors, and online learning communities whose knowledge and resources continue to make technology education more accessible.

We also appreciate the students who have chosen to invest their time and energy in developing digital skills. Their curiosity, commitment, and willingness to learn represent the kind of mindset needed to build a better technological future.

Finally, we acknowledge the creators and maintainers of the educational resources and official technical documentation referenced in this manual.

ABOUT KJV TECH

KJV TECH

Inspiring the Next Generation of Innovators.

KJV Tech is a technology-focused brand committed to raising young innovators and equipping individuals with relevant digital skills and practical technological knowledge.

Through technology education, practical training, digital projects, and innovation-driven initiatives, KJV Tech aims to encourage young people to explore technology, develop their abilities, and become active contributors to the digital world.

Our focus includes:

- Web development and software engineering education.
- Programming and digital skills training.
- Technology awareness and innovation.
- Practical learning through projects.
- Encouraging creativity, problem-solving, and entrepreneurship.

Our Vision: To inspire and empower the next generation of innovators through accessible technology education and practical digital skills.

Our Mission: To create learning opportunities that help individuals understand, explore, and apply technology to solve problems and create value.

Website: https://kjvtech.vercel.app

CONTACT AND OFFICIAL ONLINE PRESENCE

For enquiries about KJV Tech training programmes, educational resources, partnerships, and other technology-related initiatives, students and interested organisations can visit the official KJV Tech website.

Official Website: https://kjvtech.vercel.app

YouTube: KJV Tech Official – https://www.youtube.com/@kjvtechofficial001

Students are encouraged to follow KJV Tech's official platforms for educational updates, technology resources, training opportunities, and future programmes.

COPYRIGHT AND USAGE NOTICE

© 2026 KJV Tech. All rights reserved.

This manual is an educational resource developed for learning and instructional purposes under KJV Tech.

No part of this publication should be reproduced, redistributed, republished, or commercially exploited as a complete work without appropriate permission from KJV Tech.

Students may use the manual for personal study, classroom learning, and educational reference. External resources and documentation mentioned in this manual remain the intellectual property of their respective authors, organizations, and publishers.

The information in this manual is intended to provide foundational knowledge. Students should consult current official documentation when working with technologies that may have been updated.